

<https://roadmap.sh/packs/html/what-is-html>

What is HTML?

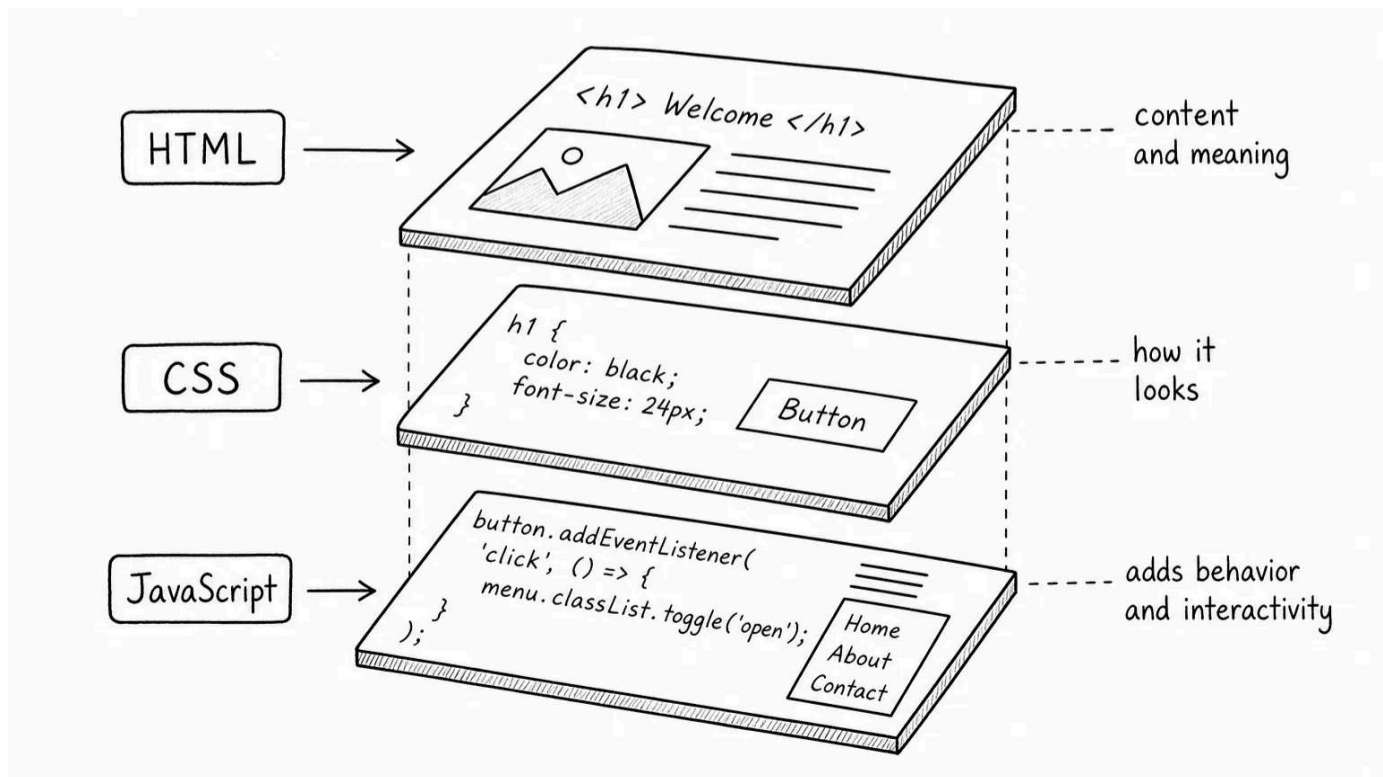
In the [browser lesson](#), you saw that when you visit a webpage, the browser's first HTTP request usually asks for an HTML document. The server sends back HTML, and the browser uses that HTML as the starting point for rendering the page.

HTML gives a webpage its content and structure. Before a page can be styled, animated, or made interactive, the browser needs HTML to know what is on the page.

At a high level, a webpage is built from a few pieces working together:

- **The server** sends HTML to the browser.
- **HTML** describes the content and meaning.
- **CSS** controls how that content looks.
- **JavaScript** adds behavior, like opening menus, reacting to clicks, validating forms, or loading new data without a full page refresh.

The details come later in the pack. For now, hold onto the basic picture: HTML is the starting structure. Everything else either produces it, styles it, or changes the DOM that the browser builds from it.



HTML stands for **HyperText Markup Language**. "Hypertext" means text that can link to other documents. The more important part for this lesson is "markup."

HTML is markup, not programming

HTML is not a programming language. It has no variables, no loops, no functions, no logic. You cannot write a calculator in pure HTML.

What HTML does is *describe*. It is plain text with tags that label parts of a document: this is a heading, this is a paragraph, this is a link, this is an image. The browser reads those labels and uses them to build the page.

A very small HTML file looks like this:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>My first page</title>
  </head>
  <body>
    <h1>Hello, web</h1>
    <p>This is a paragraph of text.</p>
  </body>
</html>
```

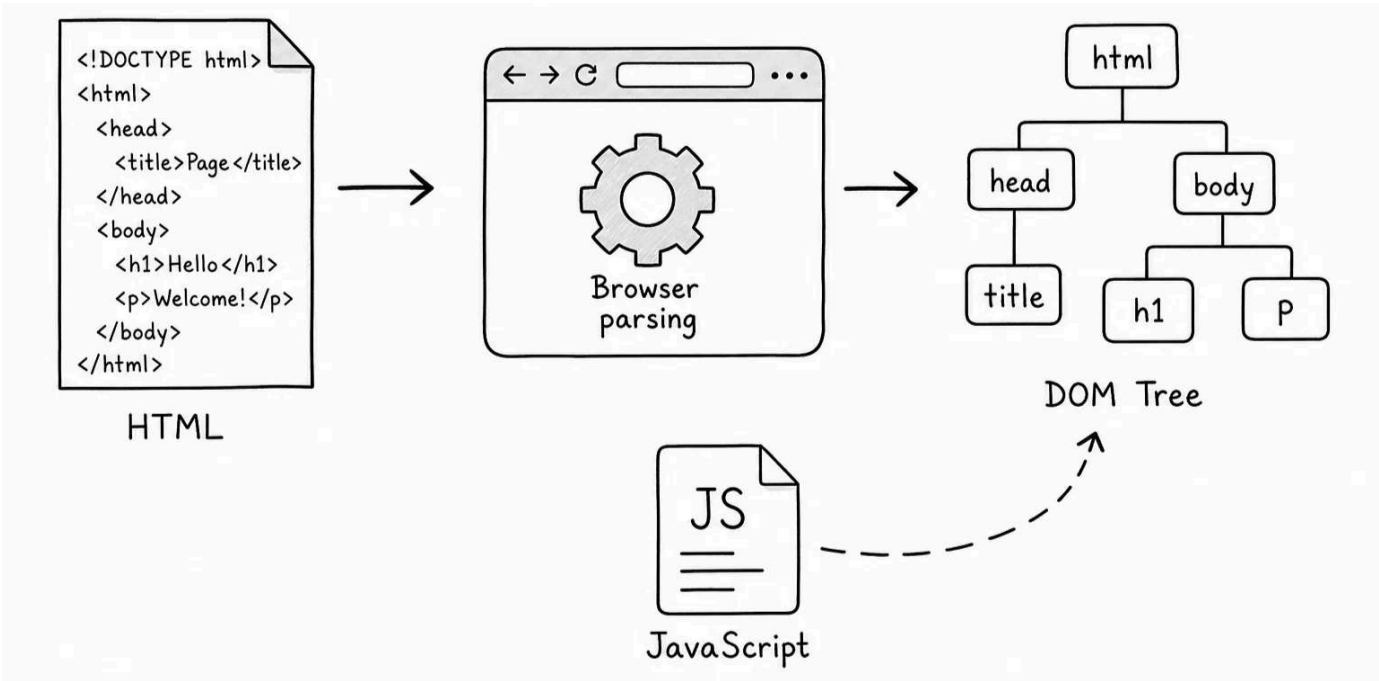
If you save that as `hello.html` and open it in a browser, you get a page with a heading and a paragraph. Nothing has been compiled. No server is running. The browser read the text in that file and figured out what to draw.

You will learn the syntax in the next lesson: what tags are, how attributes work, why some tags do not need a closing tag, and what the lines inside `head` are doing. For now, the important idea is that an HTML file is a description of structure and meaning, not a program.

HTML becomes the DOM

When the browser receives HTML, it does not just print the text on the screen. It parses the file into a tree-shaped structure called the **DOM**, short for Document Object Model.

Each element in the HTML file becomes a node in the tree. The `html` element contains the `head` and the `body`. The `body` contains headings, paragraphs, lists, images, and so on. Nested elements become nested nodes.



The DOM matters because it is what everything else on the page works with. CSS uses the DOM to find elements and apply styles. JavaScript uses the DOM to read content, react to clicks, and change what is on screen. Accessibility tools, search engines, and browser extensions look at the DOM too, not the raw HTML text.

So HTML is the input, and the DOM is what the browser actually works with.

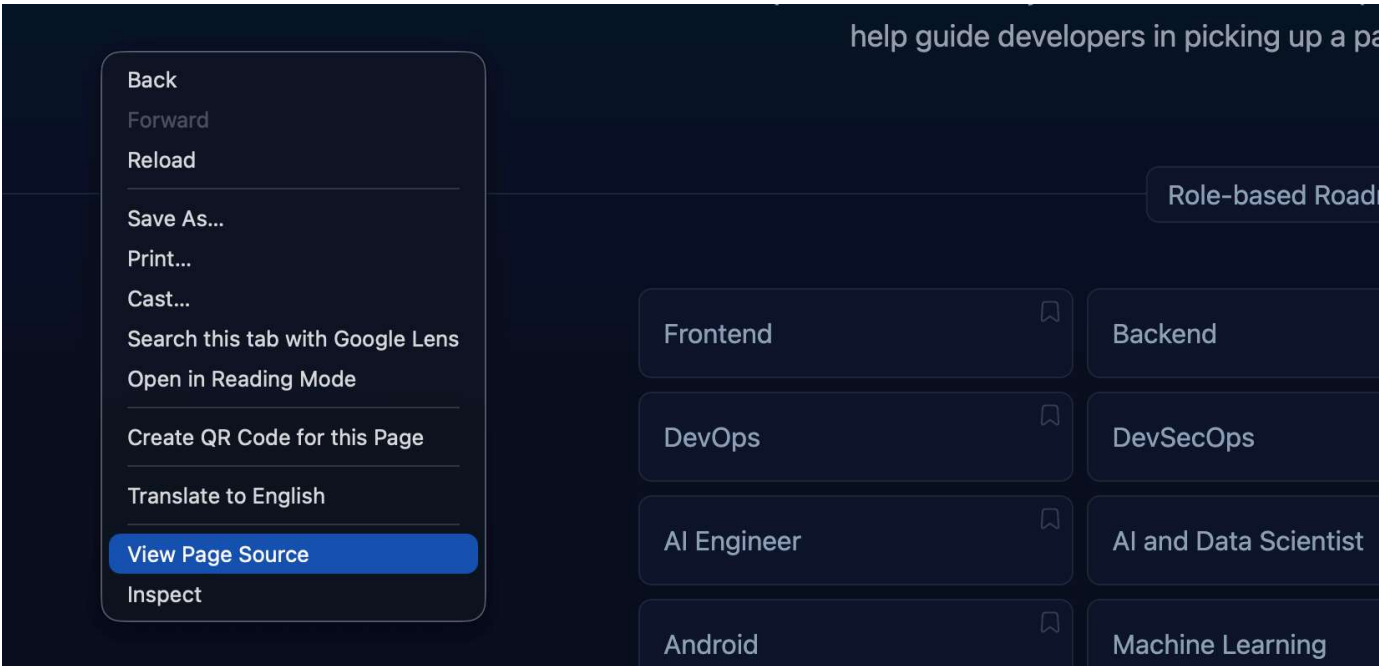
The DOM can change after the page loads

This is the detail that trips beginners up the most.

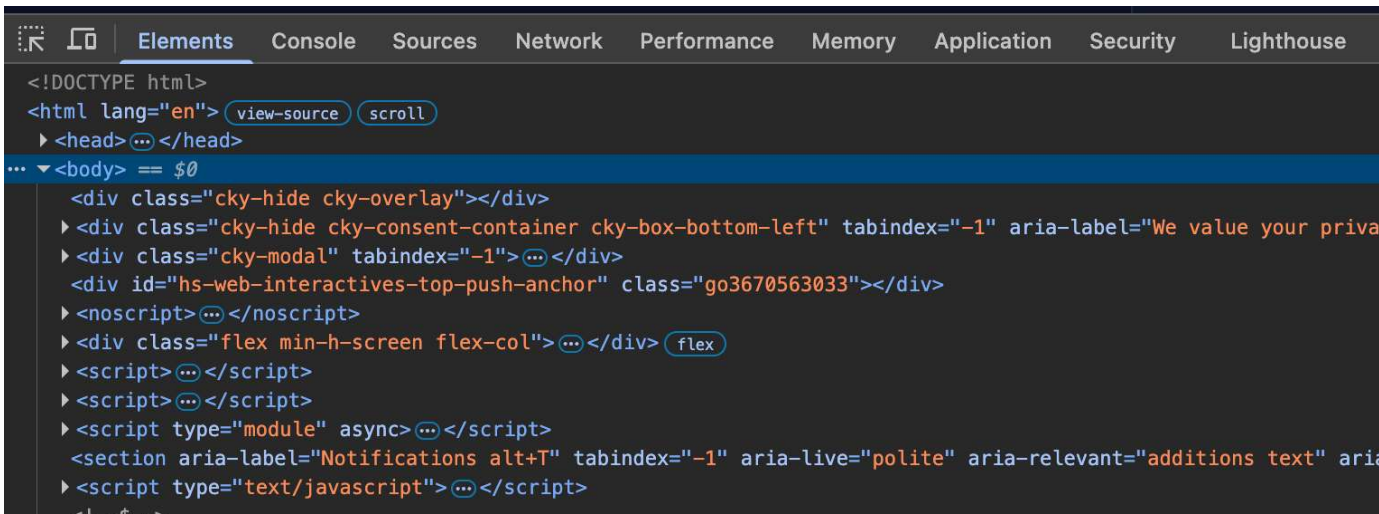
The DOM usually starts from the HTML the server sent. But once the page is loaded, JavaScript can add, remove, or change nodes in the tree. By the time you are looking at a page, the DOM may not match the original HTML response anymore.

That difference shows up in your browser's developer tools:

View Page Source shows the *original* HTML the server sent. It does not change. You can access this by right-clicking on a webpage and choosing **View Page Source**.



Elements in DevTools shows the *current* DOM. It updates as JavaScript runs.



On a simple static page, the two will look almost identical. On a modern web app, View Source might show a mostly empty HTML shell, while Elements shows a fully built-out page. That is JavaScript filling in the DOM after the initial response.

⚠️ **The DOM is not the file** - The DOM starts from the HTML response, but it is a live structure inside the browser, not a copy of the file. Confusing the two will make a lot of frontend behavior look like magic later.

You do not need to think about this every time you read HTML. Just remember that "the page" the browser shows is the DOM, not the HTML text by itself.

Where HTML shows up in real work

HTML shows up across frontend, backend, and fullstack work.

Frontend developers write HTML directly or through components. Fullstack and backend developers often produce HTML through templates, where server data is mixed with markup before being sent to the browser. Server frameworks also render HTML emails, error pages, admin dashboards, and confirmation screens.

The HTML your app sends or generates is what the browser parses, what users see, what screen readers announce, and what search engines index. It is part of the product, not a leftover detail.



Show me a small example of a server-rendered HTML page where the same template is filled with different data for two different users. Keep it beginner friendly.

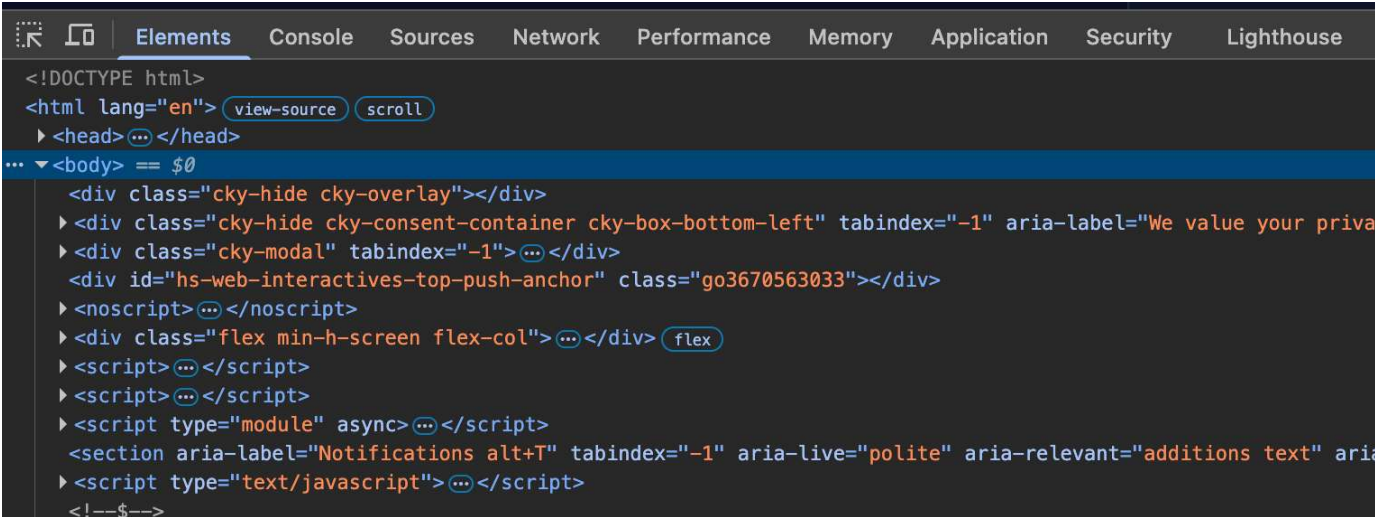
Try it

Take the following HTML and save it as **hello.html** :

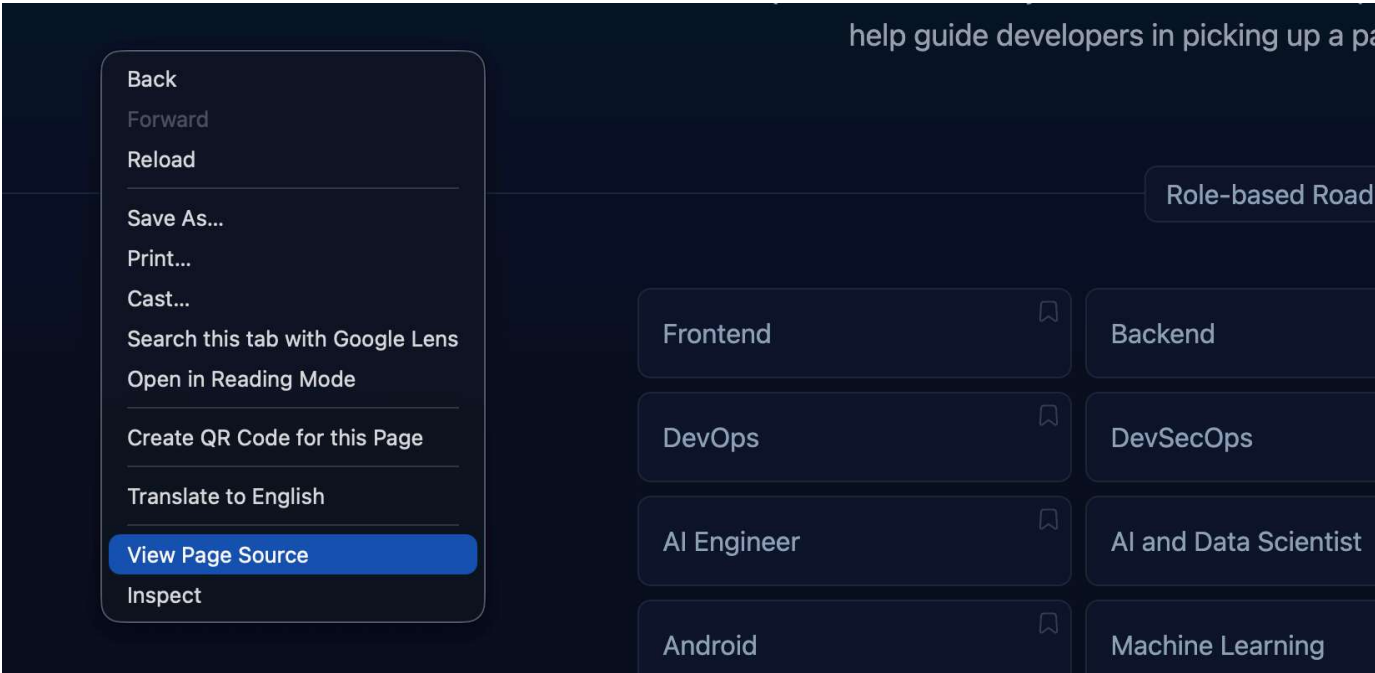

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>My first page</title>
  </head>
  <body>
    <h1>Hello, web</h1>
    <p>This is a paragraph of text.</p>
  </body>
</html>
```

Now open it in your browser. Open DevTools with **F12** , **Cmd+Option+I** , or by right-clicking and choosing **Inspect**.

Click the **Elements** tab. The tree you see there is the live DOM. You can expand and collapse nodes, hover over them to see which part of the page they correspond to, and watch them update if the page changes.



Now, on the same page, right-click anywhere and choose **View Page Source** (or **View Source**). A new tab opens with the raw HTML source. Compare it with what you saw in Elements.



For this simple file, they should look almost identical. After that, try the same thing on a modern web app. View Source and Elements may look very different there. That difference is the DOM at work.

< 0 / 6 >

Knew

0 Items

Learnt

0 Items

Skipped

0 Items

ResetProgress

Test Your Knowledge

What is HTML?

[Click to Reveal the Answer](#)

- ✔ Already Know that
- ✦ Didn't Know that
- ▶ Skip Question



LESSON COMPLETE

Nicely done.

Up next: Pricing Comparison Table

Next Lesson →

Anatomy of an HTML document

The last lesson left you with a small example file and the idea that HTML describes structure and meaning. This lesson takes that file apart and explains the pieces that show up in almost every page. By the end, you should be able to read, write, and modify the basic skeleton of an HTML document.

Every modern HTML document follows the same basic shape:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>My first page</title>
  </head>
  <body>
    <h1>Hello, web</h1>
    <p>This is a paragraph of text.</p>
  </body>
</html>
```

Each line has a job:

- `<!DOCTYPE html>` tells the browser to use modern HTML rules. It is not really a tag; it is a declaration. Without it, the browser may fall back to old compatibility behavior called quirks mode.
- `<html lang="en">` is the root element of the page. The `lang` attribute tells screen readers, search engines, and translation tools what language the content is in.
- `<head>` holds information *about* the page: its title, its character encoding, links to CSS, scripts, and so on. Users do not see this content directly.
- `<body>` holds everything users actually see and interact with: headings, paragraphs, images, forms, buttons.

The `head` and `body` are siblings inside `html`. Their jobs are different, and you should write both explicitly in normal HTML pages.

Tags, elements, and attributes

Almost everything inside an HTML document is an element. A typical element looks like this:

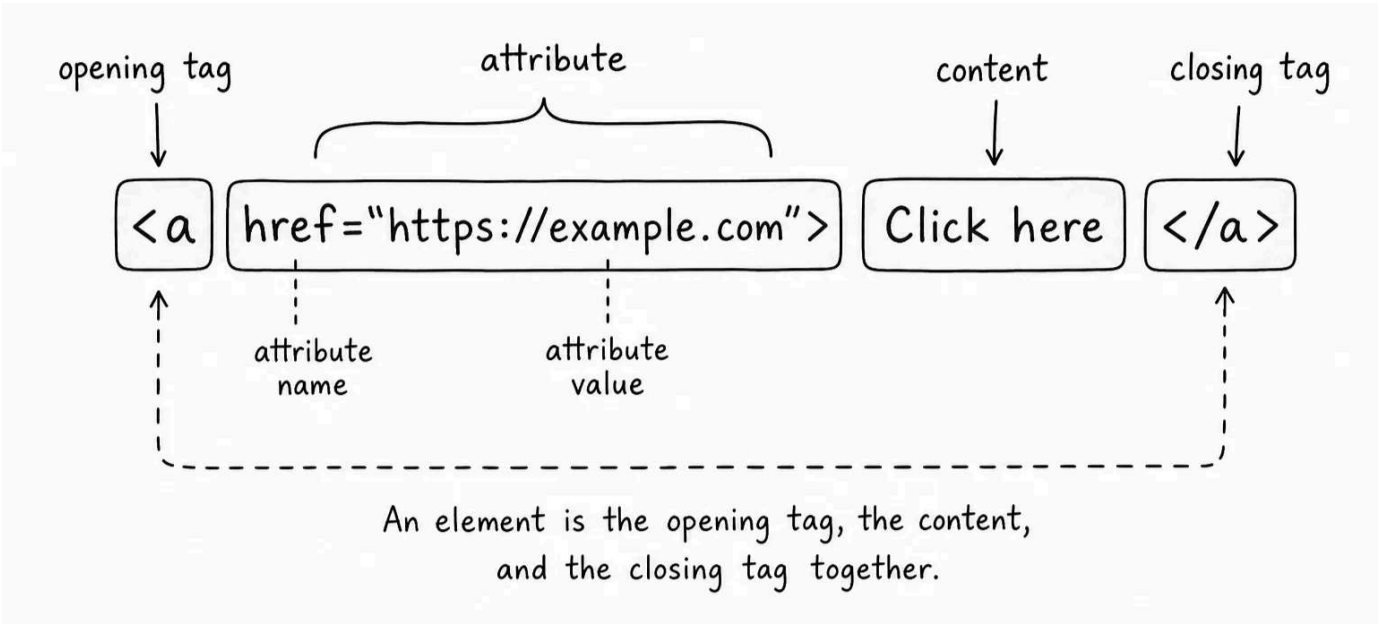
html

```
<a href="https://example.com">Click here</a>
```

There are four parts to notice:

- `<a>` is the **opening tag**. It marks the start of the element.
- `</a>` is the **closing tag**. The slash means "closing".
- `Click here` is the **content** of the element.
- `href="https://example.com"` is an **attribute**, a name/value pair inside the opening tag that gives the element extra information.

Together, the opening tag, the content, and the closing tag form one **element**. The tag is just the marker; the element is the whole thing.



Most attributes are written as `name="value"`. Single quotes work too, but double quotes are the convention. A few attributes are *boolean*: writing the name on its own is enough. For example, the `disabled` attribute on a button:

html

```
<button disabled>Submit</button>
```

You can stack many attributes on the same element:

html

```
<input type="email" name="email" required placeholder="you@example.com">
```

Three attributes that show up everywhere

Three attributes appear in almost every HTML file you will read or write: `id`, `class`, and `data-*`.

The `id` attribute is a unique label for one element on the page. Only one element should have a given id. CSS, JavaScript, and fragment links all use ids to find a specific element.

html

```
<h2 id="pricing">Pricing</h2>
```

In that snippet, the `h2` has `id="pricing"` . A URL like `https://roadmap.sh#pricing` opens `https://roadmap.sh` and scrolls to the element with that id.

The `class` attribute is a shared label. Many elements can have the same class, and one element can have many classes separated by spaces. In the example below, the paragraph has two classes: `lead` and `intro` .

html

```
<p class="lead intro">A short summary paragraph.</p>
```

Classes are mostly used by CSS to apply styles, and by JavaScript to find groups of elements.

The `data-*` attributes are for your own custom data. Any attribute starting with `data-` is treated as valid, and the browser will not complain about it.

html

```
<button data-product-id="42">Add to cart</button>
```

Browsers ignore `data-*` attributes by default. JavaScript can read them, and CSS can select elements by them when needed.

💡 `id` and `class` are your main hooks into the page. You will use them in every CSS and JavaScript lesson in the future.

Void elements and nesting

Not every element wraps content. Some elements are written as a single tag and never have a closing tag. These are called **void elements**.

html

```

<br>
<hr>
<input type="text" name="email">
<meta charset="UTF-8">
<link rel="stylesheet" href="style.css">
```

You do not write `</img>` or `</br>` . They have no content to wrap, so a closing tag would have nothing to close. You may see older examples written like `<br />` , but in HTML, `<br>` is the normal form.

Elements that *do* have content can contain other elements. This is called **nesting**. The rule is simple: elements must close in the order they opened, like balanced brackets.

This is correct:

html

<p>This text is important and clear.</p>

This is broken. The **strong** element should close before the **p** :

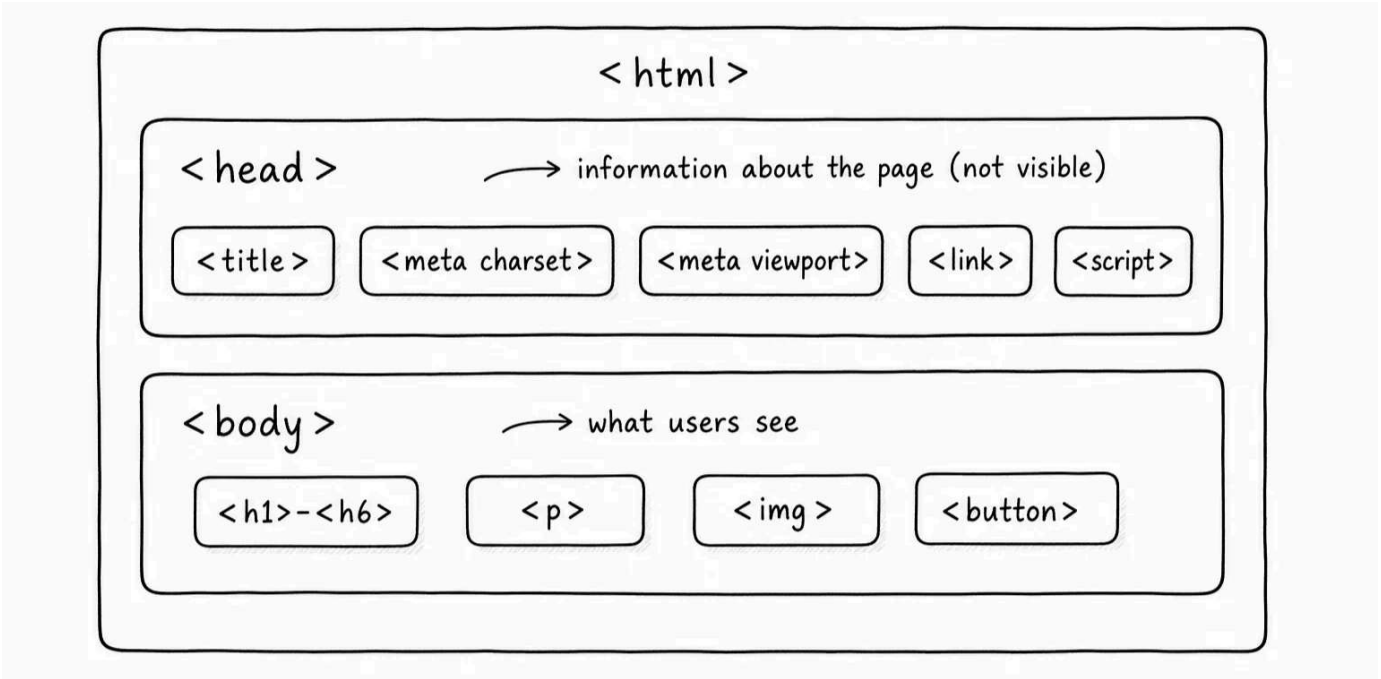
html

<p>This text is important</p>

Browsers try to fix broken nesting on their own, but the result is not reliable. Keep your tags balanced.

What goes in the head

The **head** is for information that helps the browser interpret and present the page. It is not where you put the main content users read, click, or type into.



The most important things in the head are these five.

<title> sets the text shown in the browser tab and in search results.

html

<title>Pricing - Acme</title>

<meta charset="UTF-8"> tells the browser how to decode the text in the file. UTF-8 covers virtually every character used by humans today.

 **Always set UTF-8**

Without `charset="UTF-8"` , names like `André` or `Müller` may appear as garbled characters in your users' browsers. Modern frameworks set this for you, but it is worth knowing why the line is there.

`<meta name="viewport" content="width=device-width, initial-scale=1">` tells mobile browsers to use the actual device width when

rendering. Without it, phones may try to render the page as if it were on a wide desktop and zoom out.

`<link rel="stylesheet" href="style.css">` loads a CSS file.

The `rel="stylesheet"` attribute tells the browser what kind of resource the link is pointing to.

`<script src="app.js" defer></script>` loads a JavaScript file.

The `defer` attribute tells the browser to keep parsing the HTML and run the script after the document is built. Without `defer` , the browser stops parsing to download and run the script, which slows the page down.

ⓘ Don't worry about about the `link` and `script` tags yet, we will cover them in detail when we learn about CSS and JavaScript.

That is most of what beginners need from the head. Other tags belong there too, such as social previews, favicons, and analytics, but you can pick those up as you need them.

Comments and entities

Comments are a way to add information to your HTML files that is ignored by browsers. Comments can span multiple lines; you start a comment by writing `<!--` and end with `-->` - everything between these two is ignored by the browser.

html

```
<!-- This is a comment. The browser ignores it. -->

<!--
  This is a multiline. You can
  break it down in multiple lines
  to make it easier for you to read.
-->
```

Comments are useful for leaving notes for yourself or other developers, or for temporarily hiding a block of HTML while you work on it.

HTML also has **entities** for characters that would otherwise be interpreted as HTML, or that are easier to write by name. The most common are:

- `<` for `<`
- `>` for `>`
- `&` for `&`
- `©` for `©`

You mostly need entities when you want to display a character that would otherwise look like markup. To show the text `<div>` on the page, write `<div>`. Otherwise the browser thinks you are starting a `div` element.

Have a look at the example below to get an idea:

html

```
<!-- in the example below, browser will try to parse <meta> tag -->
<p>You can use <meta> tag to tell the browser more about the page</p>

<!-- this will be fine -->
<p>You can use &lt;meta&gt; tag to tell the browser more about the page</p>
```

HTML in templates and frameworks

Hand-writing every HTML file is rare in real projects. Many apps generate HTML through templates, components, or server frameworks. A template usually looks like normal HTML with a few placeholders for data:

html

```
<title>{{ pageTitle }} - Acme</title>
```

When the server renders the page, it fills in `{{ pageTitle }}` with real data. The resulting HTML is what the browser receives.

This is why the document skeleton still matters even when a framework writes most of the HTML for you. A wrong `charset` can make user names show up as garbage. A missing viewport tag can break mobile layout. A script loaded without `defer` at the top of `head` can slow the whole page down.

Try it

Save this as `index.html` and open it in your browser:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>My first page</title>
  </head>
  <body>
```

```
<h1 id="welcome">Hello, web</h1>
<p class="lead">This is a paragraph of text.</p>
<!-- You can add more elements below. -->
</body>
</html>
```

Open DevTools and click the **Elements** tab. Expand the `<html>` node and look at what is inside. You should see the same **head** and **body** you wrote, with the elements nested inside them.

Try a few small changes and reload between each:

- Change the `<title>` and watch the browser tab update.
- Change the `<h1>` text and watch the page content update.
- Remove the `lang="en"` attribute. The page still works, but you have lost a small accessibility signal.
- Add a new attribute like `data-author="you"` to the `<h1>` . Notice that nothing visible changes, but the attribute is there in DevTools.
- Paste the file into an HTML validator and fix any mistakes it reports.

You now have a working file you can build on for the rest of this pack.

< 0 / 6 >

✓Knew 0 Items

★Learnt 0 Items

⏮Skipped 0 Items

↺ResetProgress

Test Your Knowledge

What is the difference between the head and the body?

[Click to Reveal the Answer](#)

✓Already Know that

★Didn't Know that

⏮Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Pricing Comparison Table

Next Lesson

Text and content structure

Now that you can write the skeleton, the next step is filling the HTML body with content. The most common kind of content is text i.e. headings, paragraphs, lists, quotes, and code.

HTML has tags for many common kinds of text. Browsers ship with some default styles for all the tags. However you should pick the tag that matches what the text *means*, not the one whose default style happens to look right.

Headings and the document outline

Headings give a page its structure. There are six heading levels, from `<h1>` down to `<h6>` :

html

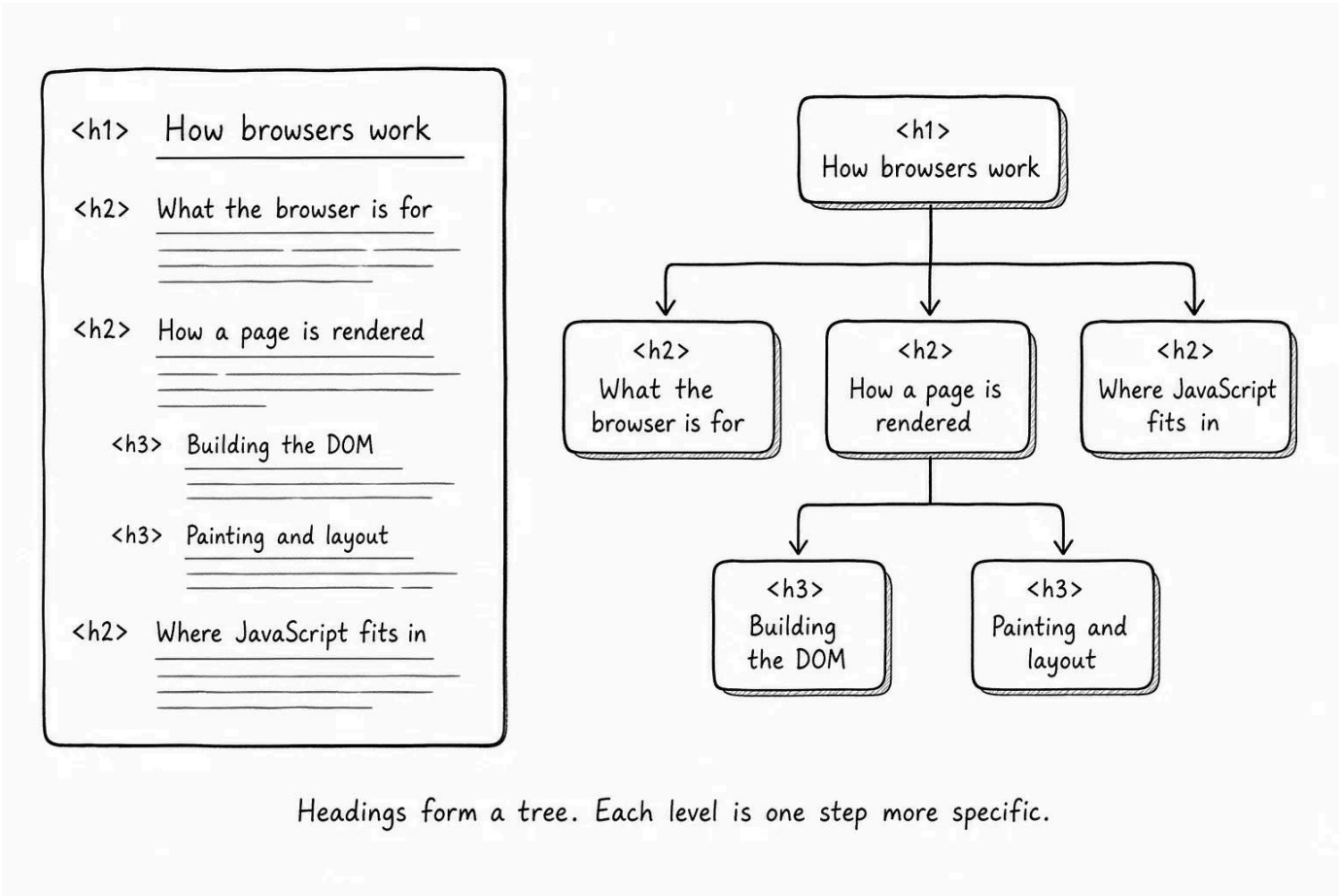
```
<h1>How browsers work</h1>
<h2>What the browser is for</h2>
<h2>How a page is rendered</h2>
<h3>Building the DOM</h3>
<h3>Painting and layout</h3>
<h2>Where JavaScript fits in</h2>
```

You can think of headings as a tree of sections. The `h1` names the whole page. Each main section is an `h2` . Inside each `h2` , you may have one or more `h3` s, and so on. The deeper you go, the more specific the topic.

A few conventions to follow:

- Use one `<h1>` per page that describes the whole page.
- Step down one level at a time. Do not jump from `<h2>` to `<h4>` .
- Use headings for real section titles, not just to make text bigger.

Hierarchy matters because screen readers, search engines, and outline tools all use it to understand the shape of the page. Skipping levels or sprinkling extra `<h2>` s to make text look bigger makes that structure harder to follow.



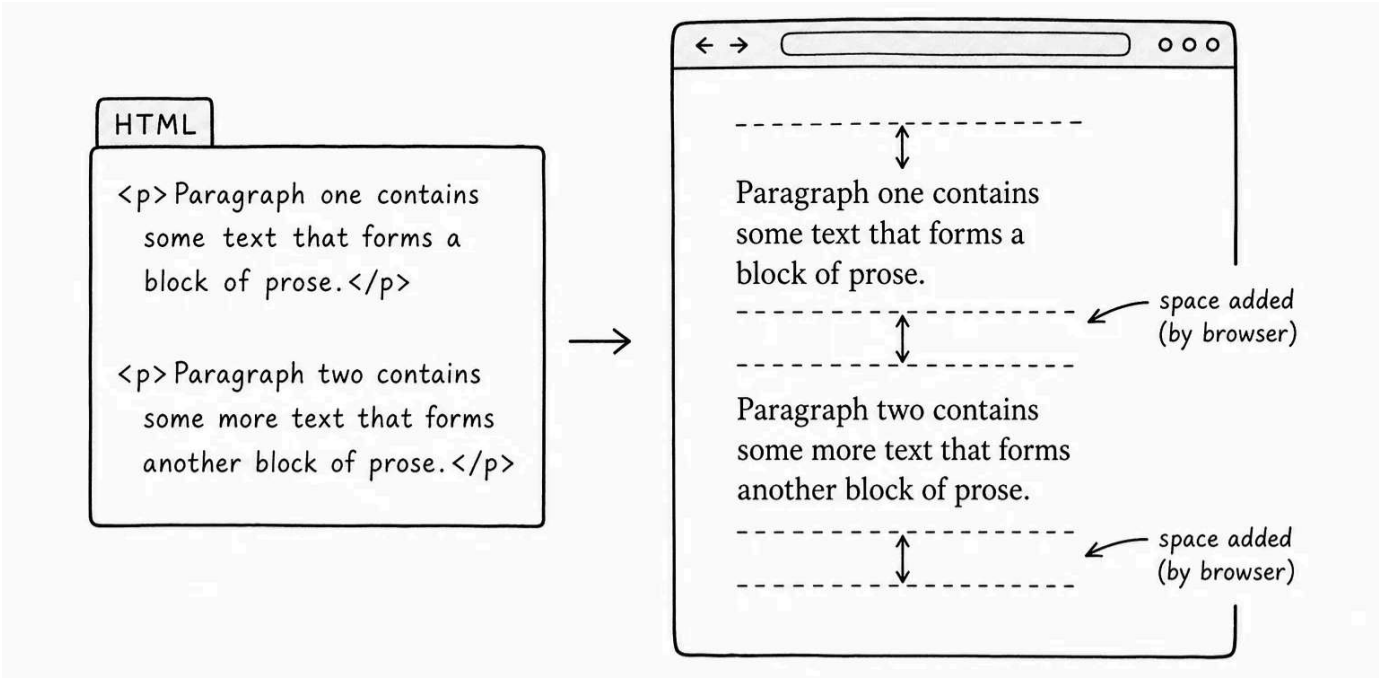
Paragraphs, breaks, and rules

A paragraph is written with the `<p>` element:

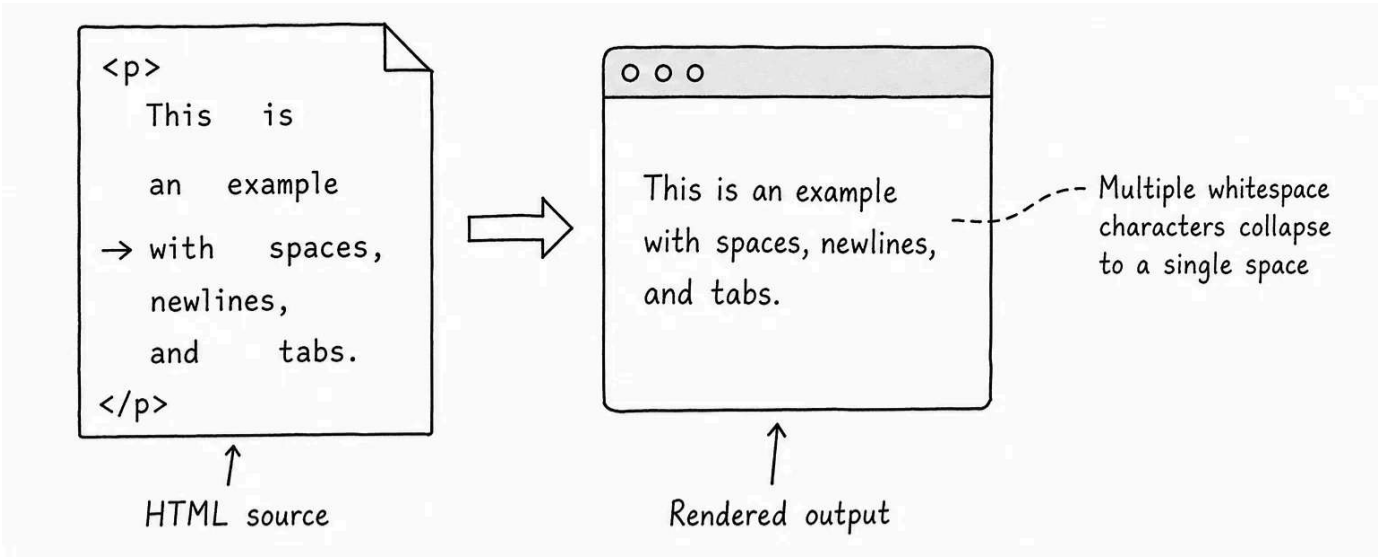
html

```
<p>HTML is a markup language. It describes the structure of a document so the browser can
<p>HTML alone is enough to put text on a page, but CSS and JavaScript usually come along
```

Paragraphs are the default container for text. The browser adds vertical space around them by default.



HTML collapses whitespace in normal text. Many spaces, tabs, and newlines in your source become a single space when rendered. That is why you do not have to format your HTML source the way you want the text to look on the page.



When you do need a line break inside a block of text, like in an address, poem, or other line-by-line text, use `<br>` :

html

```
<p>
  221B Baker Street<br>
  London<br>
  England
</p>
```

Do not use `<br>` for vertical spacing between sections. That is what paragraphs and CSS spacing are for.

The `<hr>` element draws a thematic break, usually shown as a horizontal line. Use it when you want to suggest a topic shift without adding a heading.

Emphasis and importance

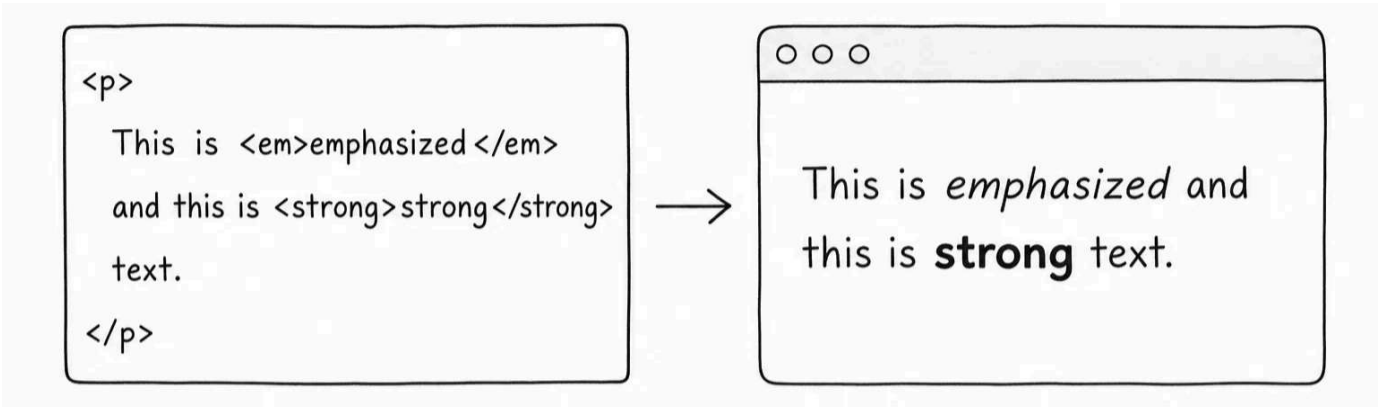
There are two elements `em` and `strong` which are used to emphasize some part of a text:

- `<em>` marks text that should be *emphasized*. Browsers render it in italics by default, but the meaning is "say this with stress when read aloud".
- `<strong>` marks text that is **important**. Browsers render it in bold by default, but the meaning is "this matters more than the surrounding text".

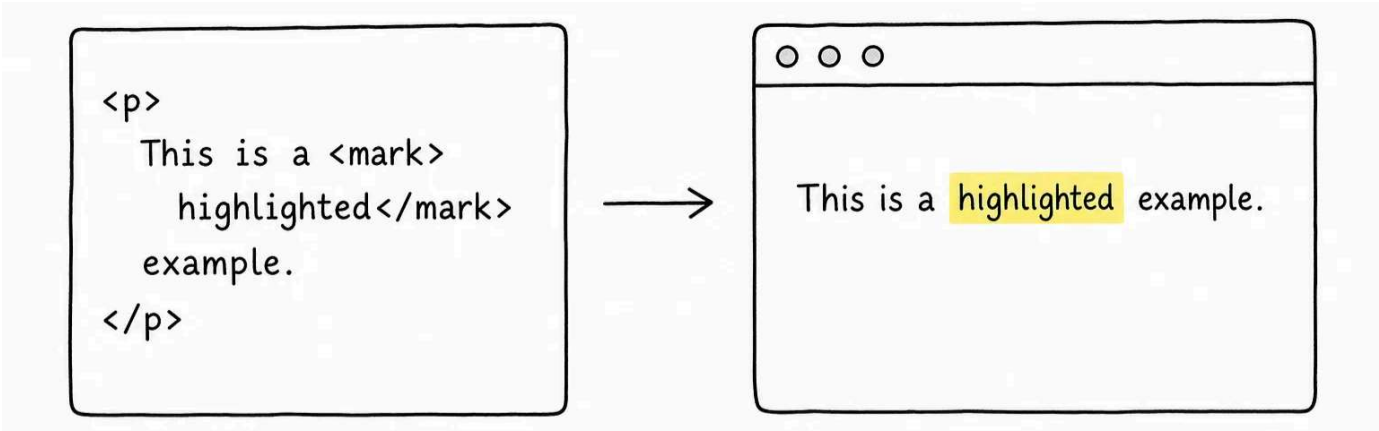
html

```
<p>You can write HTML in any text editor, but <em>save the file with a .html extension</p>
<p><strong>Never share your API keys in client-side code.</strong></p>
```

You may also see `<i>` for italic and `<b>` for bold. These still exist, but `em` and `strong` are semantically meaningful and should be preferred.



You may also use `<mark>` for text you want to highlight, like search hits. Browsers render it with a yellow background by default.



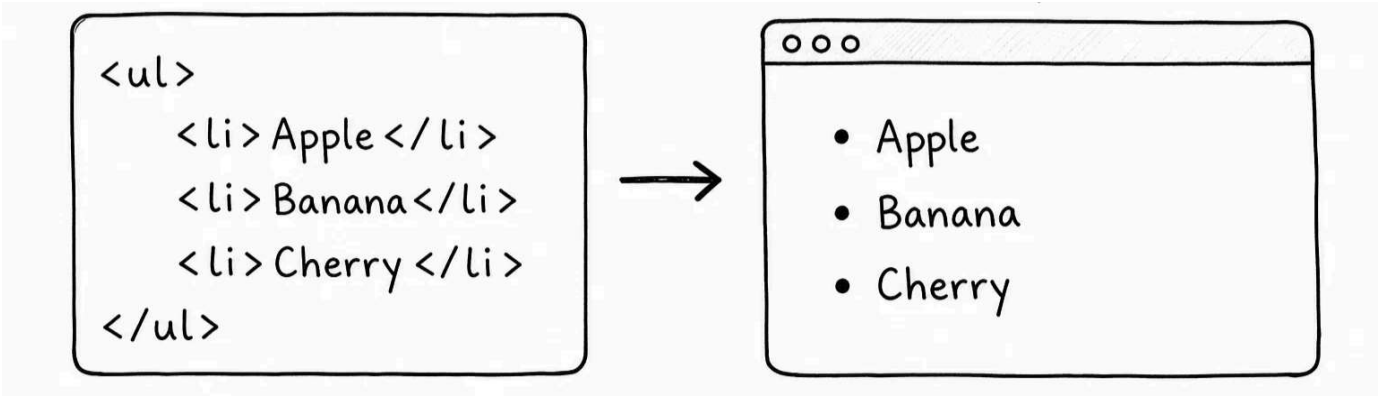
Lists

There are two main kinds of lists, plus a less common third.

`<ul>` is an unordered list. Use it when the items are a group without a meaningful order.

```
html

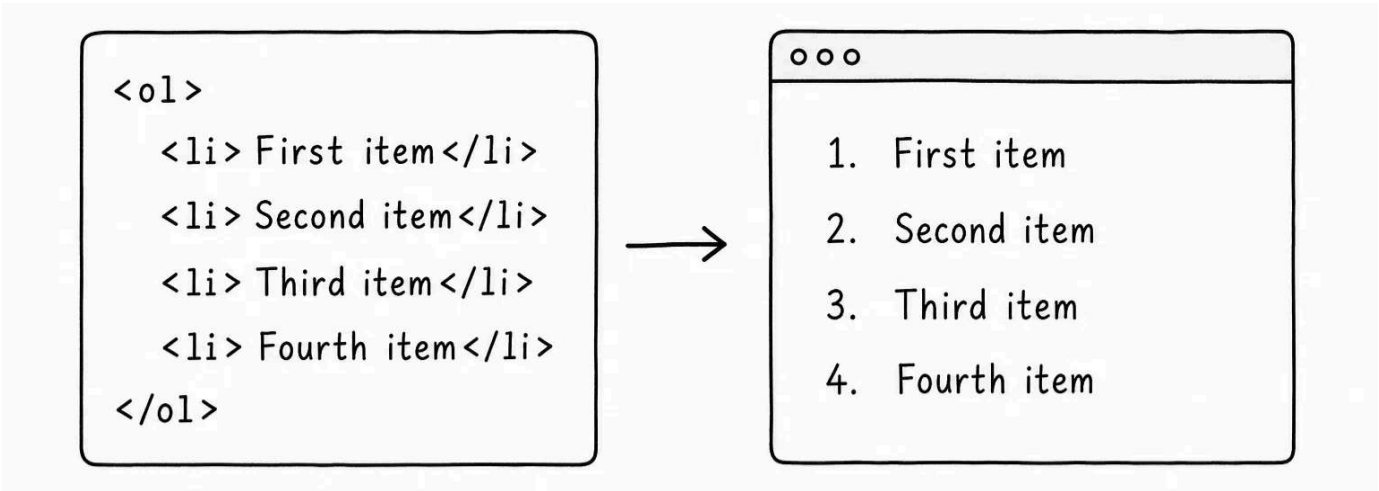
<ul>
  <li>Coffee</li>
  <li>Tea</li>
  <li>Water</li>
</ul>
```



`<ol>` is an ordered list. Use it when order matters, like steps in a recipe or items in a ranking.

```
html

<ol>
  <li>Open the editor.</li>
  <li>Create a file named index.html.</li>
  <li>Paste the skeleton.</li>
  <li>Open it in your browser.</li>
</ol>
```



Lists can be nested. Put a `<ul>` or `<ol>` inside an `<li>` to create sub-items:

html

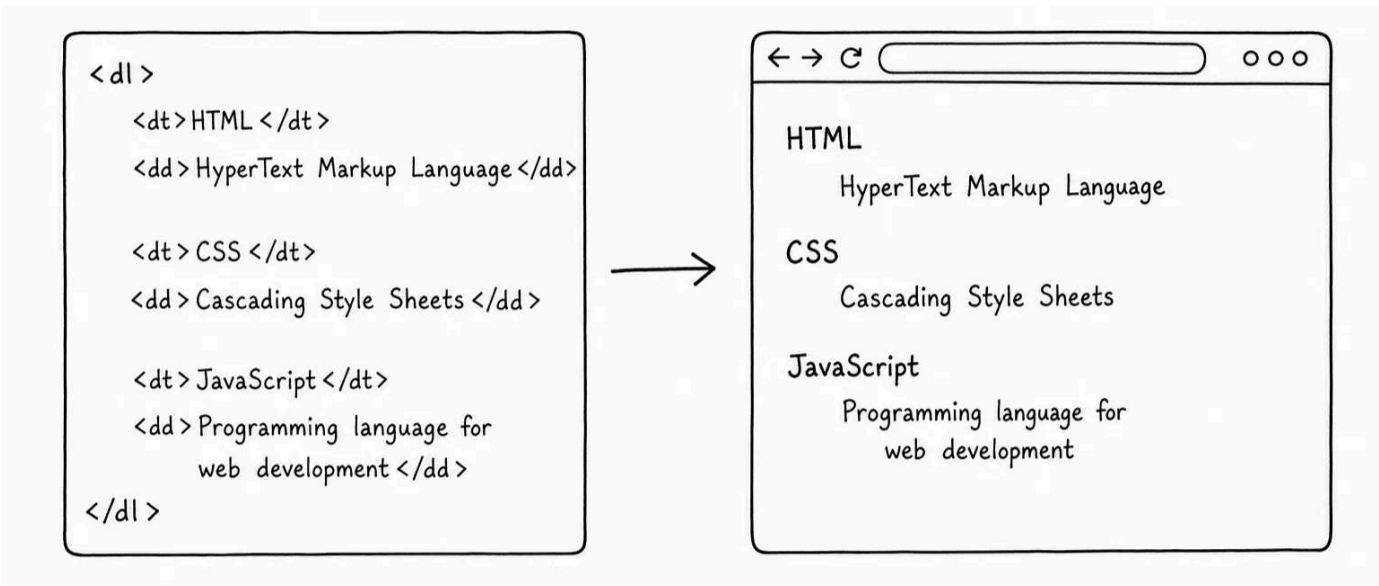
```
<ul>
  <li>Drinks
    <ul>
      <li>Coffee</li>
      <li>Tea</li>
    </ul>
  </li>
  <li>Snacks</li>
</ul>
```

There is also `<dl>` , a description list, used for name and value pairs like glossaries:

html

```
<dl>
  <dt>HTML</dt>
  <dd>A markup language for describing the structure of web pages.</dd>
  <dt>DOM</dt>
  <dd>A tree of nodes the browser builds from HTML.</dd>
</dl>
```

`<dt>` is the term, and `<dd>` is its description. You will see description lists less often than `<ul>` and `<ol>` , but they are the right choice for terms and definitions.



Quotes

Two elements mark quoted text i.e. `blockquote` and `q` .

`<blockquote>` is for longer, block-level quotes. Browsers usually indent it from the surrounding text.

html

```
<blockquote>
  <p>This paragraph is quoted from another source.</p>
</blockquote>
```

`<q>` is for short, inline quotes. Browsers add quotation marks for you, so you do not type them yourself.

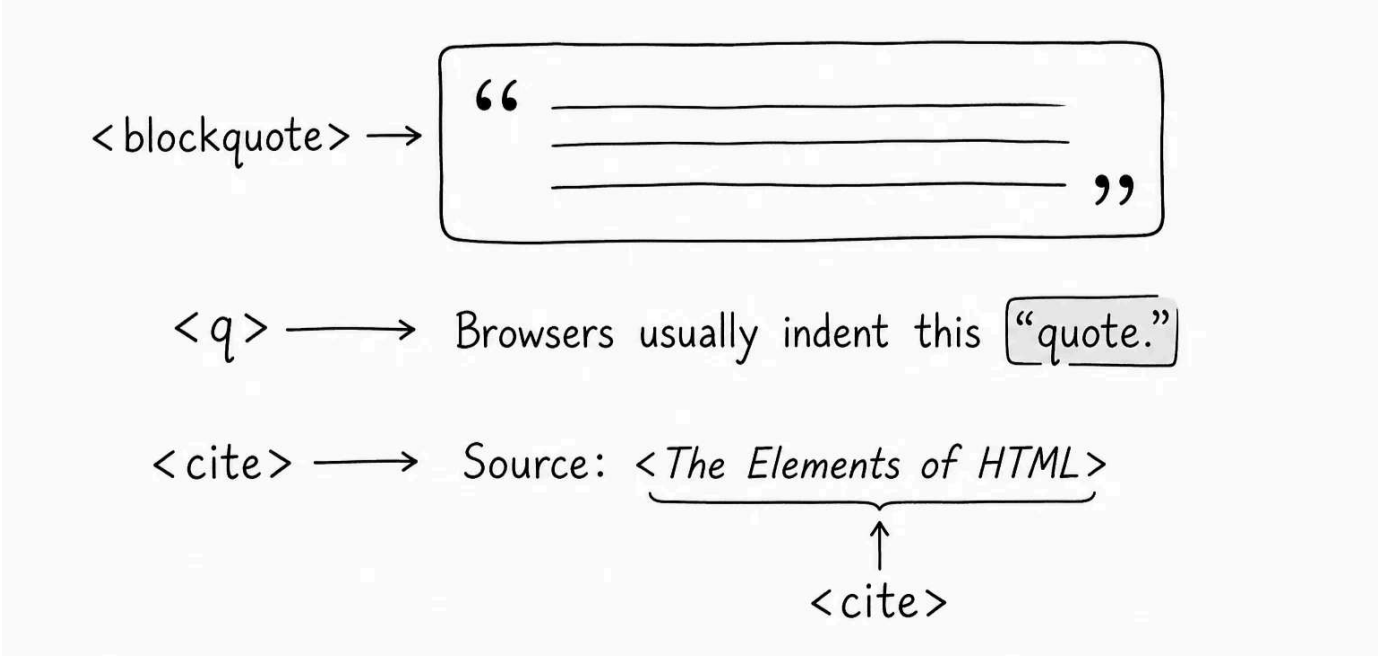
html

```
<p>She said <q>let us ship the page already</q> and closed the laptop.</p>
```

To credit the source, the `<cite>` element wraps the title of a work, such as a book, article, or paper.

html

```
<blockquote>
  <p>A clear document explains its structure before it asks the reader to remember detail
  <p><cite>A Guide to Web Documents</cite></p>
</blockquote>
```



Code in HTML

Two elements work together when you want to show code on a page.

`<code>` marks a piece of inline code. It is for short fragments inside running text. Browsers render it in a monospace font.

html

```
<p>The <code>defer</code> attribute tells the browser to keep parsing the HTML.</p>
```

`<pre>` is for preformatted blocks of text. Inside `<pre>`, the browser preserves whitespace exactly as you wrote it.

html

```
<pre>function greet(name) {
  return "Hello, " + name;
}
</pre>
```

The common combination is `<pre><code>` for code blocks:

html

```
<pre><code>function greet(name) {
  return "Hello, " + name;
}</code></pre>
```

The `<pre>` preserves indentation. The `<code>` signals that the content is source code.

Meaning over appearance

The same principle keeps coming back: pick the tag for what the text *means*, not for how it looks.

It is tempting to use `<h6>` for a small subtitle because it looks small, or `<blockquote>` to indent a paragraph because the indent is convenient. CSS lets you style anything to look any way you want. The semantic tag is what tells the browser, the screen reader, the search engine, and the next developer what the text actually is.

💡 **Stay semantic** - If you ever catch yourself picking a tag because of how it looks, swap it for a tag that matches the meaning, and use CSS for the look.

When text comes from users

If your app shows text that came from a user, two things are worth knowing now.

⚠️ **Escape user content** - Escape special characters before inserting user input into your HTML. If a user types `<script>` into a comment field and your server inserts it raw, the browser will treat it as a real script tag. Most template engines escape values by default, but it is worth confirming.

Do not reach for `<br>` every time user text contains newlines. If the text is code or terminal output, a `<pre>` block may be right. If it is normal prose,

render the escaped text in a paragraph and use CSS such as `white-space: pre-wrap` when you want newlines to display.

Try it

Create a file called `about.html` and build a short "about me" page using only the elements from this lesson:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>About me</title>
  </head>
  <body>
    <h1>Your name here</h1>
    <p>One short paragraph introducing yourself.</p>

    <h2>What I'm learning</h2>
    <ul>
      <li>HTML basics</li>
      <li>...</li>
    </ul>

    <h2>A quote I like</h2>
    <blockquote>
      <p>Put your favorite quote here.</p>
    </blockquote>

    <h2>Tools I use</h2>
    <ol>
      <li>...</li>
    </ol>
  </body>
</html>
```

Open the file in your browser and add a few real items to the lists. Wrap a word in `<em>` and another in `<strong>` inside one of the paragraphs. Open DevTools and inspect the structure. Notice that the page is readable and accessible with no CSS at all because the browser's defaults already give you a workable document.

< 0 / 6 >

Knew 0 Items

Learnt 0 Items

Skipped 0 Items

ResetProgress

Test Your Knowledge

What is the rule of thumb for picking an HTML tag?

[Click to Reveal the Answer](#)



Already Know that



Didn't Know that



Skip Question



LESSON COMPLETE

Nicely done.

Up next: Pricing Comparison Table

Next Lesson

Links, URLs, and navigation

Links are what turn a collection of independent files into a navigable site, and they are how the web ties itself together. This lesson is about the `<a>` element and the URLs that go inside it.

The anchor element

A link in HTML is written with the `<a>` element. The address you want to link to goes in the `href` attribute, and the visible text goes between the opening and closing tags.

html

```
<a href="https://example.com">Example site</a>
```

When the user clicks the link, the browser navigates to whatever the `href` points to. The default behavior is to replace the current tab with the new page.

The `<a>` element can also wrap images, headings, or other content. Anything inside the tags becomes clickable. Do not put another link or a button inside a link; nested interactive elements are confusing for browsers and assistive tools.

html

```
<a href="/about">
  
</a>
```

Use links for navigation: going to another page, another site, or another place on the same page. Use `button` tag (covered in later lessons) for actions that happen on the current page, such as opening a menu, submitting a form, or dismissing a dialog. A link with `href="#"` just to catch a click is usually a sign that you wanted a button.

Absolute, root-relative, and relative URLs

The value of `href` can take a few different shapes depending on where the link is going.

Absolute URLs include the full protocol and host. Use them when linking to a different site.

html

```
<a href="https://example.com/articles/intro">External article</a>
```

Root-relative URLs start with `/` and are interpreted from the root of the current site. Use them for internal links across an app.

html

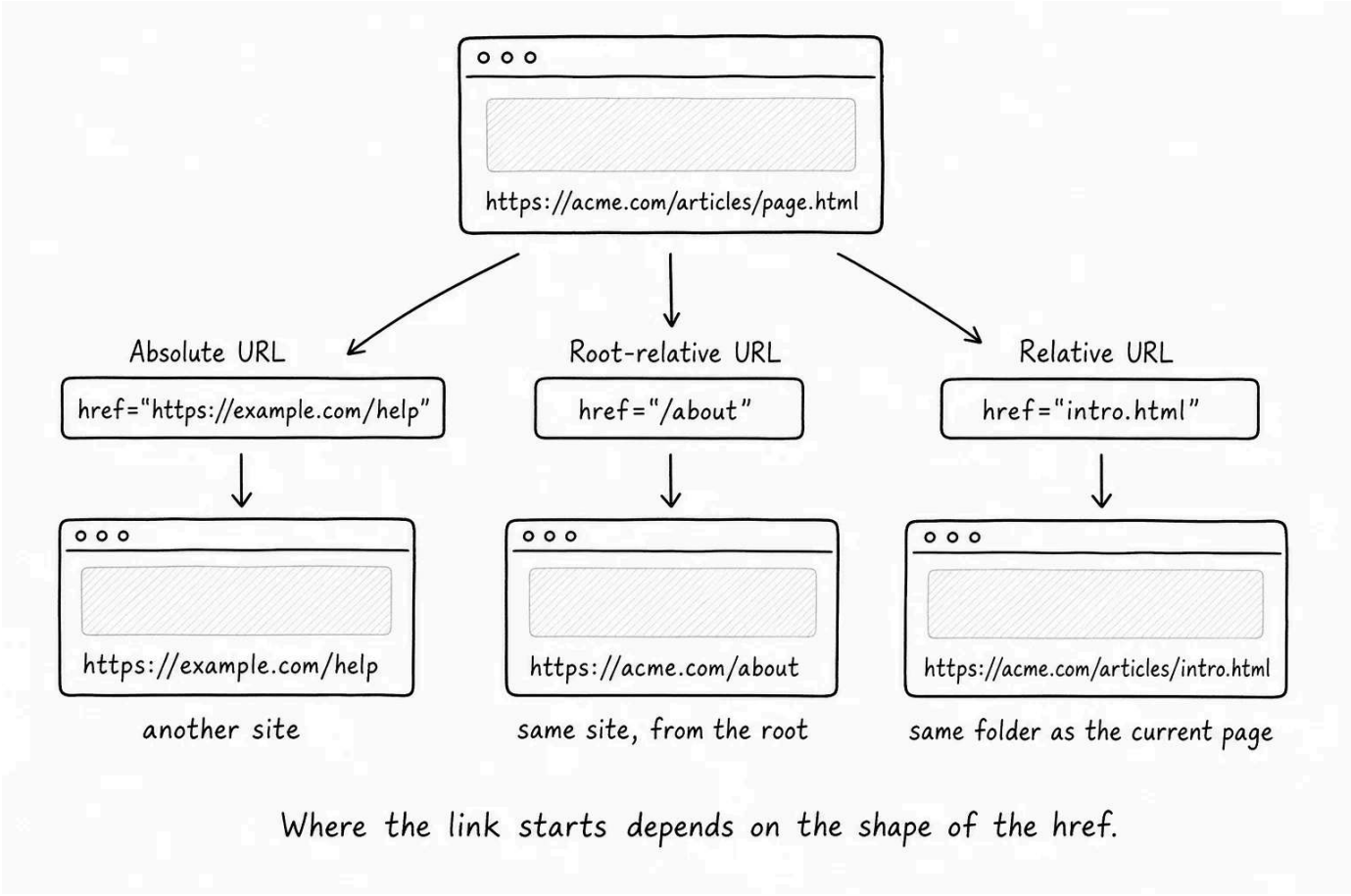
```
<a href="/articles/intro">Intro article</a>
```

A link with `href="/articles/intro"` from `https://acme.com/about` points to `https://acme.com/articles/intro`.

Relative URLs are interpreted from the current page's location. Use them when linking between sibling files.

html

```
<a href="about.html">About</a>
<a href="articles/intro.html">Intro article</a>
<a href="../index.html">Back to home</a>
```



For real apps, **root-relative URLs** are usually the right default for in-app links. They keep working when you move a page, and they do not depend on the current page's location.

Fragment links

A URL can include a `#` followed by an id. The browser uses that to jump to an element with the matching id on the page.

html

```
<a href="#pricing">Jump to pricing</a>
```



```
...
...
...
<h2 id="pricing">Pricing</h2>
```

Clicking the link scrolls the page to the **Pricing** heading. Fragment links work inside the same page or across pages:

html

Meet the team

That link goes to **/about** and scrolls to the element with **id="team"**.

Opening links in new tabs

By default, a link replaces the current tab. To open the link in a new tab, set **target="_blank"**.

html

External site

The **rel="noopener"** attribute is good practice whenever you use **target="_blank"**. It prevents the new page from getting a JavaScript reference back to the page that opened it, which is a small but real security and performance improvement. Modern browsers add this behavior automatically, but writing it is still the convention. You will learn more about this in the future lessons.

You may also see **rel="noreferrer"**, which goes further by hiding the referring URL from the destination. Use it when you do not want the destination to know where the click came from.

💡 **Default to same-tab** - Open links in the same tab unless there is a real reason not to. Users can always open a new tab themselves; forcing it removes that choice and takes the click out of the normal back-button flow.

Link text matters

A link is a navigation choice. The text inside the link should tell the user where it goes.

Avoid generic text:

html

```
<a href="/pricing">click here</a> to see our plans.
```

Prefer text that describes the destination:

html



```
See our <a href="/pricing">pricing plans</a>.
```

Why it matters:

- **Screen readers** can list every link on a page. A list of "click here, click here, click here" is useless.
- **Search engines** use link text as a signal for what the destination is about.
- **Scanning** the page is easier when link text describes where each link goes.

If a link contains only an image, the image's **alt** text becomes the link text for screen reader users. That is one more reason useful alt text matters.

mailto and tel links

A few special URL schemes start a different kind of action when clicked. **mailto:** opens the user's email client; **tel:** starts a phone call on capable devices.

html



```
<a href="mailto:hello@example.com">Email us</a>
<a href="tel:+12025550123">Call us</a>
```

Both are useful as shortcuts, but they depend on the user's device having an email or phone app configured. Do not rely on them as the only contact path.

When URLs come from app code

URL structure is one of the places where HTML, routing, and server code meet.

A few small habits pay off later:

- **Use root-relative URLs** for in-app links in most apps.
Hardcoding **https://yourapp.com/about** breaks in local development, staging, and preview environments.
- **Generate URLs from your router** instead of building them by hand in templates (i.e. if your backend code handles the page routing). Renaming a route then updates every link automatically.

- **Pick stable URL paths.** Once a link is out in the world, breaking it is expensive. Bookmarks, emails, and search results all point at it.

Try it

Make a two-page site by creating two files in the same folder.

index.html :

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Home</title>
  </head>
  <body>
    <h1>Home</h1>
    <p>This is the homepage. Read more <a href="about.html">about this site</a>.</p>
  </body>
</html>
```

about.html :

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>About</title>
  </head>
  <body>
    <h1 id="top">About</h1>
    <p>This page tells you about the site.</p>
    <h2 id="contact">Contact</h2>
    <p>Get in touch at <a href="mailto:you@example.com">you@example.com</a>.</p>
    <p><a href="#top">Back to top</a> · <a href="index.html">Back to home</a></p>
  </body>
</html>
```

Open **index.html** and click the link. On the about page, click the fragment link, then "back to home", and watch the URL change in the address bar. You are seeing the two most common local-file links: **about.html** points to another file in the same folder, and **#top** points to an element on the current page.

< 0 / 6 >

Knew 0 Items

Learnt 0 Items

Skipped 0 Items

ResetProgress

Test Your Knowledge

What is the href attribute?

[Click to Reveal the Answer](#)

- ✔ Already Know that
- ✦ Didn't Know that
- ▶ Skip Question



LESSON COMPLETE

Nicely done.

Up next: Pricing Comparison Table

Next Lesson

Images, media, and embedded content

The web is more than text. Pages show photos, diagrams, videos, audio, and content embedded from other sites. HTML has dedicated elements for these, and most of them follow the same basic pattern: tell the browser where the media lives, then provide useful text for people or browsers that cannot use the media directly.

That useful text looks different depending on the element. For images, it is usually `alt` text. For video and audio, it can be fallback text between the opening and closing tags. For iframes, it is a `title` that labels the embedded page.

Images: the `<img>` element

The `<img>` element loads an image into the page. It is a void element, which means it has no content and no closing tag. It relies on two essential attributes.

html

``

The `src` attribute is the path to the image. It can be an absolute URL, a root-relative path, or a relative path.

The `alt` attribute is text that describes the image. Leaving out `src` means the image will not load. Leaving out `alt` means users who cannot see the image may lose the information it carries.

Alt text: write it for someone who cannot see the image

Alt text is what screen readers read aloud, what search engines can use, and what the browser shows when the image fails to load. Good alt text describes the image's purpose in the context of the surrounding content.

For an informative photo:

html

``

For an image that conveys data, like a chart or diagram:

html

```

```

For a purely decorative image, one that adds visual flair but no information, use an empty `alt` attribute. Screen readers will skip it.

html

```

```

Empty `alt` is intentional. Leaving the attribute off entirely is different: assistive tools then have nothing to work with and may read the filename instead. Always include the `alt` attribute.

💡 **Write alt like a caption** - If the image were replaced by your alt text, would the page still make sense? If yes, your alt text is doing its job.

HTML source	What a sighted user sees	What a screen reader announces
<pre></pre>		image: A black cat sitting on a windowsill

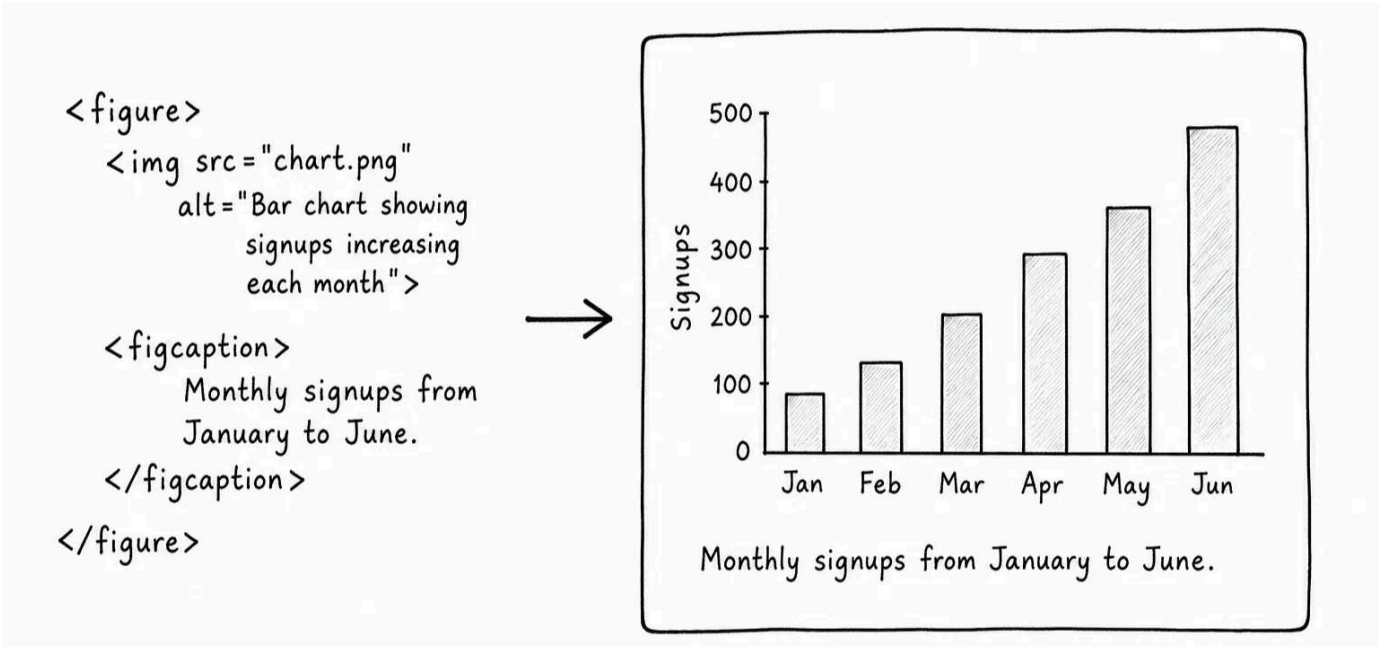
Figures and captions

When an image needs a visible caption, wrap it in `<figure>` and put the caption in `<figcaption>` .

html

```
<figure>
  
  <figcaption>Monthly signups from January to June.</figcaption>
</figure>
```

The `alt` text and the caption are not the same job. The `alt` text stands in for the image when someone cannot see it or when the image fails to load. The caption is visible page content that gives extra context to everyone.



Dimensions and layout shift

Set the width and height of an image when you know them:

html

```

```

These attributes do two things. They reserve space in the layout while the image is downloading, which prevents the page from jumping around when the image arrives. They also give the browser the image's aspect ratio so CSS can scale it without distortion.

You can still resize the displayed image with CSS. The attributes are about reserving space, not locking the pixel size.

If an image is far down the page, you can also add `loading="lazy"` :

html

```

```

This tells the browser it can wait to load the image until the user gets closer to it. Do not use lazy loading for the main image at the top of the page, because that image should load right away.

Image formats

The format you choose affects file size and qkuality.

Format	Best for
JPG / JPEG	Photographs and rich images with smooth gradients
PNG	Logos, screenshots, or images that need transparency
WebP	A modern format that often beats JPG and PNG on file size

Format	Best for
SVG	Icons, logos, and diagrams that should scale crisply at any size
GIF	Short looping animations (often better as a video today)

Browsers load all of these the same way. A safe beginner default: JPG for photos, PNG for screenshots, SVG for icons.

Responsive images, briefly

Sometimes you need to serve different image sizes to different screens. A phone does not need the same huge hero image as a large desktop monitor. HTML has `srcset` and `<picture>` for this:

html

```

```

You do not need the details yet. Just know these exist for responsive images. They become useful when image performance starts to matter.

★ AI Tutor

Explain responsive images and srcset with a small beginner example. >

Video and audio

The `<video>` and `<audio>` elements work similarly to `<img>`, but unlike `<img>`, they have content and a closing tag.

html

```
<video src="intro.mp4" controls width="640" height="360" poster="intro-cover.jpg">
  Your browser does not support video playback.
</video>
```

Two attributes matter right away.

- `controls` tells the browser to show play, pause, and volume controls. Without it, the video has no visible controls.
- `poster` is a placeholder image shown before the video starts playing.

The text between the tags is fallback content, shown only when the browser cannot play the video at all.

For audio, the pattern is similar but simpler:

html

```
<audio src="podcast.mp3" controls>
  Your browser does not support audio playback.
</audio>
```

If you need to provide more than one file format, use `<source>` elements inside the video or audio:

```
html

<video controls>
  <source src="intro.webm" type="video/webm">
  <source src="intro.mp4" type="video/mp4">
</video>
```

The browser picks the first source it can play.

Embedded content with `<iframe>`

An `<iframe>` lets you embed another HTML document inside the current one. Common examples include a YouTube video, a map, or a payment widget from a third-party service.

```
html

<iframe src="https://www.youtube.com/embed/abcd1234"
  width="560"
  height="315"
  title="Intro video">
</iframe>
```

For iframes, `title` plays the accessibility role that `alt` plays for images. It gives screen readers a useful label for the embedded content.

 **Be careful what you embed** - An iframe loads another page inside yours. Only embed content from sources you trust. Browsers add some isolation around iframes, but they cannot make an untrusted embedded page safe.

You probably will not write many iframes by hand. When you do, it is usually a snippet given to you by a service like YouTube, Vimeo, or Stripe. Paste it, give it a meaningful `title`, and move on.

When media comes from users or app data

Real apps often deal with media that users upload or that the server serves back later. A few habits matter here:

- **Validate uploaded images on the server**, not just in the browser. Check file type, size, and dimensions before storing. The `accept` attribute on a file input is a UX hint, not a security boundary.

- **Strip image metadata** when users upload photos. EXIF data can include GPS coordinates and device details that the uploader may not want shared.
- **Set cache headers** on media. Images, video, and audio can be expensive to download. A long **Cache-Control: max-age** on fingerprinted filenames is the usual approach.

☆ AI Tutor

What is EXIF data in images? >

Try it

Save any image as **photo.jpg** in a folder. Then save this file as **media.html** in the same folder and open it in your browser:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Media practice</title>
  </head>
  <body>
    <h1>Media practice</h1>

    <video src="sample.mp4" controls width="480" poster="photo.jpg">
      Your browser does not support video playback.
    </video>
  </body>
</html>
```

Try a few changes and reload between each:

- Change **alt** to an empty string **""**. Nothing visible changes for you, but a screen reader will now skip the image.
- Remove **width="320"** from the image. The image now shows at its natural size.
- Remove **controls** from the video. The player UI disappears even though the video element is still there.
- Change **poster** to point at a different image, or delete the attribute. Notice what the player looks like before it would start playing.
- Change the fallback text inside **<video>**. You probably will not see it in a modern browser, but it is still part of the HTML for browsers that cannot play the video.

You do not need a real **sample.mp4** file for most of these changes to be visible. The player UI is created from the HTML attributes. Open DevTools and

inspect the `<img>` and `<video>` nodes. Both are DOM elements with attributes, just like the elements you have already seen.

< 0 / 8 >

✔Knew0 Items

✦Learnt0 Items

▶|Skipped0 Items

↺ResetProgress

Test Your Knowledge

What are the two required attributes on an image?

[Click to Reveal the Answer](#)

✔Already Know that

✦Didn't Know that

▶|Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Pricing Comparison Table

Next Lesson

Tables and tabular data

Tables are for data that makes sense in rows and columns. Think of an orders listing, weekly schedule, a price list, plan comparisons, sports standings, or financial figures. If the relationship between rows and columns matters, use a table.

For a long time, developers also used tables to arrange whole pages. That is not how layouts are built now. CSS handles layout; tables are for data.

The easiest test is this: can you give each column a label, and does each row describe one item using those same labels? If yes, a table is probably a good fit. If you are only trying to place boxes next to each other, use CSS layout instead.

The basic structure

A minimal table looks like this:

html

```
<table>
  <tr>
    <th>Plan</th>
    <th>Price</th>
    <th>Users</th>
  </tr>
  <tr>
    <td>Starter</td>
    <td>$9</td>
    <td>1</td>
  </tr>
  <tr>
    <td>Team</td>
    <td>$29</td>
    <td>5</td>
  </tr>
</table>
```

The pieces are:

- `<table>` is the container for the whole thing.
- `<tr>` is a table row.
- `<th>` is a header cell. Browsers usually render it bold and centered, but the meaning is "this cell labels a row or column".
- `<td>` is a data cell. This is where the table's values go.

Cells go inside rows. Rows go inside the table. That is the whole shape.

Plan	Price	Users
Starter	\$9	1
Team	\$29	5

The important part is the relationship between the cells. In the example above, **Starter** , **\$9** , and **1** are not three random values. They are one row about the same plan. The headers explain what each value means.

Captions, head, and body

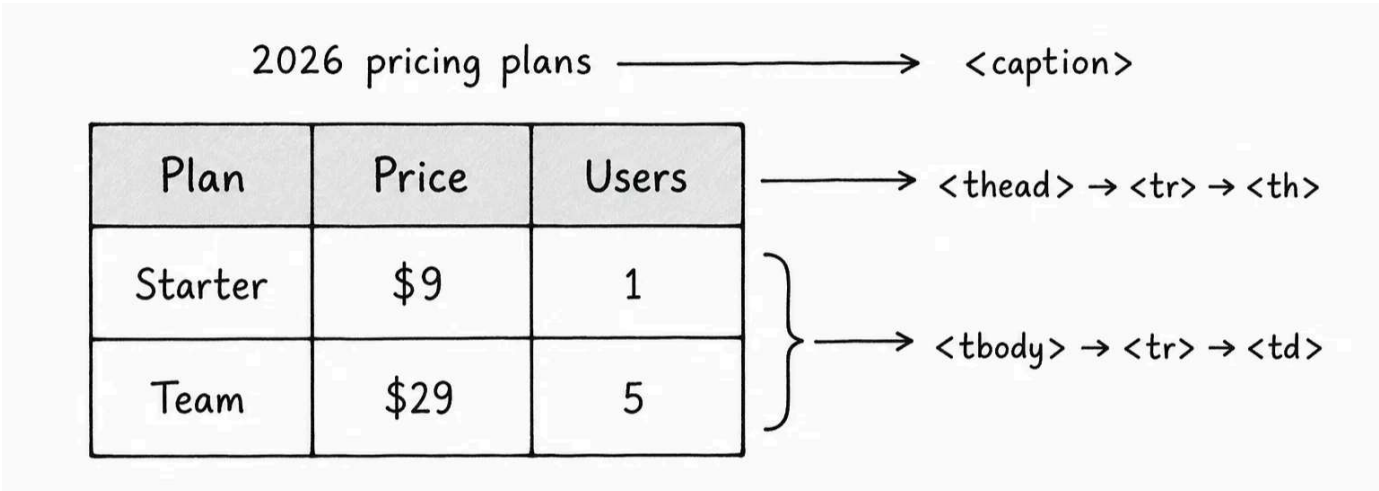
A practical table usually benefits from three more elements.

<caption> is the table's title. It belongs directly inside **<table>** as the first child.

<thead> and **<tbody>** group the header rows and data rows. You can make a table without them, but adding them makes the structure clearer and easier to style, print, scan, and maintain. There is also **<tfoot>** for a footer row, often used for totals.

html

```
<table>
  <caption>2026 pricing plans</caption>
  <thead>
    <tr>
      <th>Plan</th>
      <th>Price</th>
      <th>Users</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Starter</td>
      <td>$9</td>
      <td>1</td>
    </tr>
    <tr>
      <td>Team</td>
      <td>$29</td>
      <td>5</td>
    </tr>
  </tbody>
</table>
```



`<thead>` and `<tbody>` do not make the table look dramatically different on their own. Their job is structure. They help browsers, screen readers, CSS, and future maintainers understand which rows are labels and which rows are data.

The scope attribute

When a header cell labels a column, set `scope="col"`. When it labels a row, set `scope="row"`. This makes the relationship between headers and data cells explicit.

html

```
<thead>
  <tr>
    <th scope="col">Plan</th>
    <th scope="col">Price</th>
  </tr>
</thead>
<tbody>
  <tr>
    <th scope="row">Starter</th>
    <td>$9</td>
  </tr>
</tbody>
```

For very simple tables, that relationship is often easy to infer. Once row headers enter the picture, setting `scope` explicitly is the clearer habit.

This is especially useful for screen reader users. Instead of hearing a long stream of disconnected values, they can understand each data cell in relation to its row and column headers.

Spanning cells

A cell can span more than one column or row.

html

```
<tr>
  <th colspan="2">Pricing</th>
</tr>
```

`colspan="2"` makes this cell take up two columns. `rowspan="3"` would make a cell span three rows. Use these when the table really needs a merged

cell, such as a section header above two columns, not as a styling shortcut.

```
<table>
  <tr>
    <th colspan="3">Monthly Report</th>
  </tr>
  <tr>
    <th>Product</th>
    <th>Q1</th>
    <th>Q2</th>
  </tr>
  <tr>
    <td rowspan="2">Product A</td>
    <td>120</td>
    <td>150</td>
  </tr>
  <tr>
    <td>130</td>
    <td>160</td>
  </tr>
</table>
```

→

Monthly Report		
Product	Q1	Q2
Product A	120	150
	130	160

⚠ **Keep tables simple** - Complex tables with multiple header levels, merged cells, or nested sections can become hard to read and hard to make accessible. Use `colspan` and `rowspan` when the data truly needs them, but prefer a simpler table when you can.

Tables are for data, not layout

It used to be common to build page layouts with tables: invisible grids of rows and columns holding navigation, content, and ads. That causes problems:

- **Tables describe data relationships.** Screen readers and search engines treat tables as data. If you use them for layout, the structure no longer matches the content.
- **Layout changes are hard inside tables.** Responsive design, where the page rearranges itself on phones, is awkward to express with table cells.
- **CSS layout tools exist now.** Flexbox and grid handle every layout job tables used to be stretched to cover.

Reserve `<table>` for data that actually has rows and columns.

Good table use:

- A pricing comparison where each row is a plan and each column is a feature.
- A schedule where each row is a day and each column is a detail about that day.
- A report where each row is a record and each column is a field.

Poor table use:

- A page header with a logo on the left and navigation on the right.
- A grid of product cards.
- A two-column article layout.

When tables come from app data

Real apps often generate tables from data: admin dashboards, reports, exports, invoices, schedules, and search results.

A few habits worth picking up:

- **Include a caption** for tables generated dynamically. It gives the table a clear purpose before the user starts navigating the cells.
- **Group header rows clearly.** Use `<thead>` for headers. Printing, sticky-header CSS, and future styling are easier to manage when header rows are grouped clearly.
- **Avoid rendering huge tables all at once.** Paginate, filter, or summarize large datasets instead of sending thousands of rows in one response. Huge tables are slow to load and hard to use.

Try it

Build a small weekly schedule. Save this as `schedule.html` and open it in your browser:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Weekly schedule</title>
  </head>
  <body>
    <h1>This week</h1>

    <table>
      <caption>Study schedule</caption>
      <thead>
        <tr>
          <th scope="col">Day</th>
          <th scope="col">Topic</th>
          <th scope="col">Hours</th>
        </tr>
      </thead>
      <tbody>
        <tr>
          <th scope="row">Monday</th>
          <td>HTML basics</td>
          <td>2</td>
        </tr>
        <tr>
          <th scope="row">Tuesday</th>
          <td>Links and forms</td>
          <td>3</td>
        </tr>
        <tr>
```

```
<th scope="row">Wednesday</th>
<td>Practice project</td>
<td>2</td>
</tr>
</tbody>
</table>
</body>
</html>
```

Add a few more rows. Try changing one `<th scope="row">` to a `<td>`. The day no longer acts like a row label; it becomes just another value. Open DevTools and expand the `<table>` node in Elements. You should see the same `thead`, `tbody`, and `tr` structure you wrote.

< 0 / 6 >

✔Knew

0 Items

☆Learnt

0 Items

⏮Skipped

0 Items

↺ResetProgress

Test Your Knowledge

When should you use a table?

[Click to Reveal the Answer](#)

✔Already Know that

☆Didn't Know that

⏮Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Pricing Comparison Table

Next Lesson

Forms and user input

Forms are how users send data to a server. Signups, logins, search boxes, file uploads, etc all use forms in one way or another. Underneath the variety, a form does one simple thing: it collects a list of name and value pairs and sends them somewhere.

Forms are the most interactive part of plain HTML, and one of the few places where the browser, the network, and the server all meet on the same page.

The form element

A form wraps the controls a user fills in. Two attributes set up the trip the data takes.

html

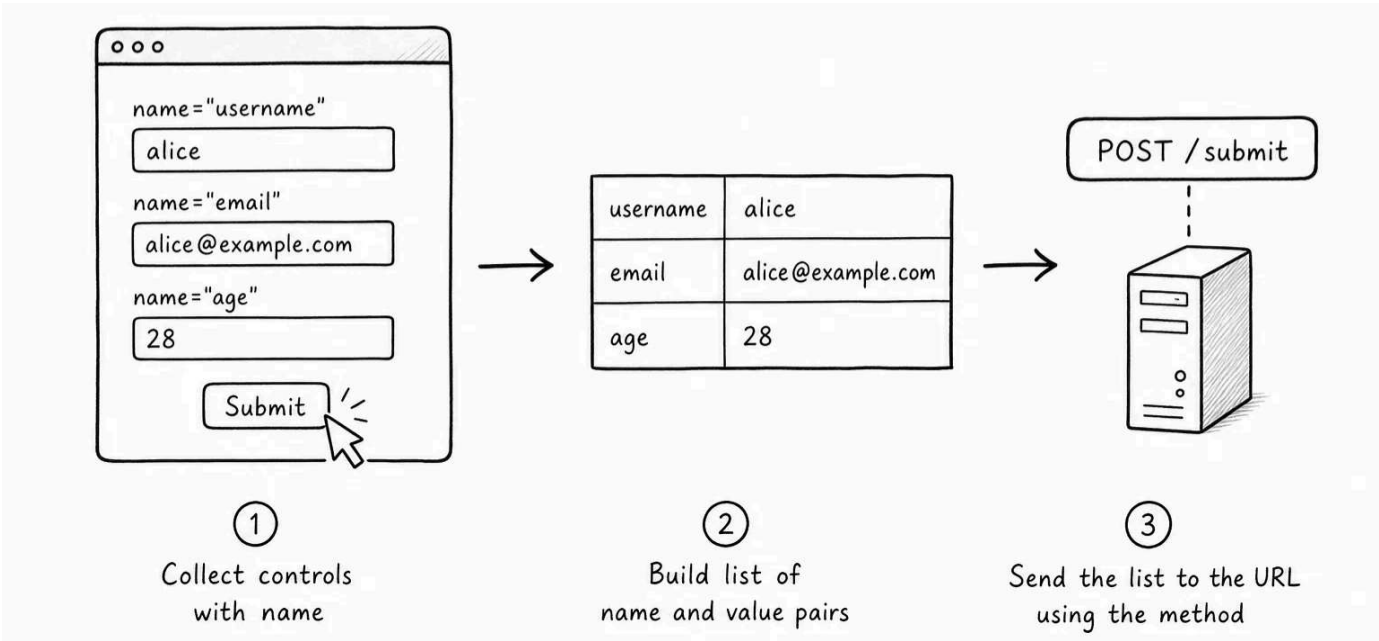
```
<form action="/signup" method="post">
  ...
</form>
```

- **action** is the URL the form sends its data to. If you don't provide it, the form submits data back to the current page's URL.
- **method** is the HTTP method used. The two normal choices in plain HTML are **get** and **post** .

When the user clicks the submit button, the browser does three things in order:

1. It collects every form control that has a **name** attribute.
2. It builds a list of name and value pairs from those controls.
3. It sends that list to the URL in **action** , using the method in **method** .

That sequence is the whole form lifecycle. Everything else in this lesson is about what each step looks like and how to control it.



GET vs POST: where the data goes

The `method` attribute decides how the form data travels to the server.

A `GET` form puts the data into the URL as a query string:

text

GET /search?q=html&page=2

This is right for things like searches, filters, and pagination. The data is short, not sensitive, and the URL can be bookmarked or shared. Reloading the page just runs the same query again.

A `POST` form puts the data into the request body instead:

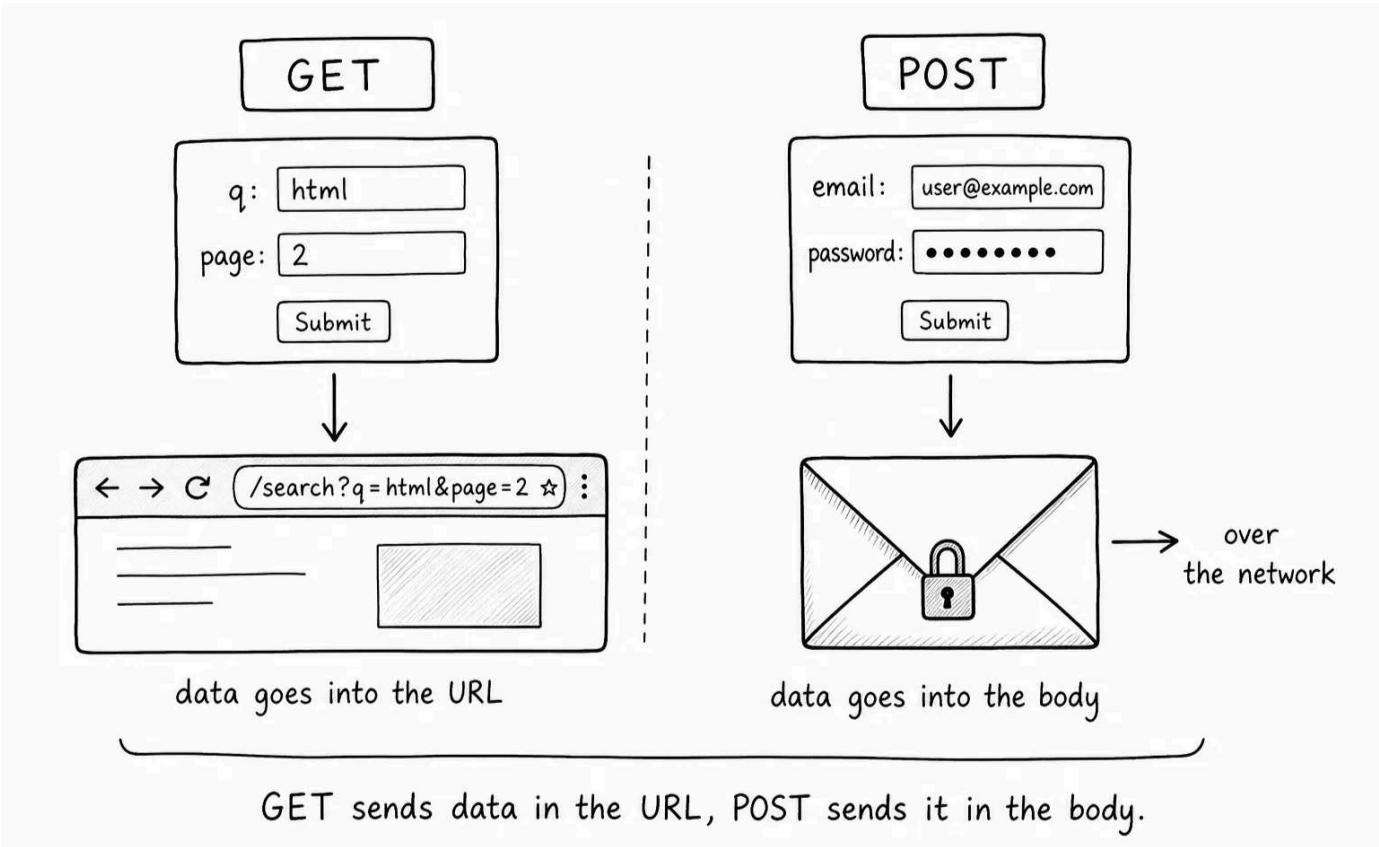
text

POST /signup
Content-Type: application/x-www-form-urlencoded

email=alice%40example.com&password=secret123

This is right for anything that changes data on the server: signups, logins, comments, settings updates, etc. The data is no longer in the URL, which keeps passwords and other private values out of browser history, link previews, and server access logs.

A rough rule of thumb: use `GET` when the form is asking for information, and `POST` when the form is making something happen.



There are other HTTP methods, like **PUT** , **PATCH** , and **DELETE** , but plain HTML forms only natively support **GET** and **POST** . The others are reached through JavaScript or through framework-specific overrides, which you will meet later.

 **POST is not encryption** - **POST** keeps form data out of the URL, but it does not encrypt the request. HTTPS is what protects the data while it travels over the network. Any form that sends passwords, payment details, private messages, or personal data should be served over HTTPS.

Inputs, names, and values

The core building block inside a form is the `<input>` element.

html

```
<input type="text" name="username" value="">
```

Three attributes do the work.

type is the kind of input: **text** , **email** , **password** , **checkbox** , and so on. It controls what UI the browser shows and what data the browser will accept.

name is the key the server receives. This attribute is the most important one on any input. If you leave **name** off, the input is not submitted at all. The browser will not include it in the name and value pairs. A missing **name** is one of the most common reasons a form "submits" but the server appears to receive nothing.

value is the input's current data. For text inputs, it is the initial text. For checkboxes and radio buttons, it is the value the server gets if the user selects

that option. As the user types or clicks, the live value updates in the DOM.

When the form is submitted, the browser builds pairs that look roughly like this:

text

username=alice
email=alice@example.com

If two inputs share the same `name`, the server receives both values under that key. That is how multiple-choice patterns like grouped checkboxes and multi-select dropdowns work.

Labels and why they pair with inputs

Every input should have a label that tells the user what it is for. Use the `<label>` element with a `for` attribute that points to the input's `id`:

html

<label for="email">Email address</label>
<input id="email" name="email" type="email">

There are two valid ways to write the pairing.

The first is the `for` and `id` pattern shown above. The `for` attribute on the label matches the `id` on the input.

The second is to wrap the input directly inside the label:

html

<label>
 Email address
 <input type="email" name="email">
</label>

Both work. The `for` and `id` pattern gives you more freedom to position the label and input separately with CSS, which is why it is the more common choice in real apps.

Labels matter for three reasons:

- **Click target.** Clicking a label focuses or activates the paired input. That makes small controls like checkboxes much easier to hit on touch screens.
- **Accessibility.** Screen readers announce the label when the input is focused. Without a label, they may only announce the input type, which tells the user nothing.

- **Clarity.** Labeled forms communicate more clearly to every user, even before any styling.

 **A placeholder is not a label** - The `placeholder` attribute looks like a label, but it disappears as soon as the user starts typing. Always pair an input with a real `<label>` , and use `placeholder` only for short examples or hints inside the field.

Input types

The `type` attribute changes what kind of value an input collects and how the browser presents it.

Type	Use it for
<code>text</code>	Free-form short text
<code>email</code>	Email addresses (with light browser validation)
<code>password</code>	Sensitive text, hidden as the user types
<code>number</code>	Whole or decimal numbers, with up and down arrows
<code>tel</code>	Phone numbers
<code>url</code>	URLs (with light browser validation)
<code>date</code>	A date, with a date picker
<code>time</code>	A time of day
<code>checkbox</code>	A standalone on/off toggle
<code>radio</code>	One choice from a group of options
<code>file</code>	A file upload (covered later in this lesson)
<code>hidden</code>	Data the user does not see or edit, but the server should still receive

Modern browsers adapt the UI to the type. For example, on a mobile device, `type="email"` shows an email-optimized keyboard, and `type="number"` shows a numeric keypad. Picking the right type is one of the easiest ways to make a form feel right on mobile.

For groups of related radio buttons, give every option the same `name` . The browser then treats the group as a single choice and submits only the selected value:

html

```
<fieldset>
  <legend>Choose a plan</legend>
  <label><input type="radio" name="plan" value="starter"> Starter</label>
  <label><input type="radio" name="plan" value="team"> Team</label>
  <label><input type="radio" name="plan" value="business"> Business</label>
</fieldset>
```

The `<fieldset>` element groups related controls, and `<legend>` is the label for the whole group. Screen readers announce the legend when the user enters the fieldset, which gives the choice some context. Use a fieldset whenever a set of inputs only makes sense together: an address block, a pair of date fields, a group of radios.

A `hidden` input is useful for passing along data the user does not need to see or edit, like a record ID or a CSRF token. It is still visible in the DOM and on the network, so it is not for secrets. Use it only for values your server wants back with the form.

Beyond `<input>` : `textarea`, `select`, and `button`

Three more elements complete the form vocabulary.

`<textarea>` is for multi-line text:

html

```
<label for="bio">Short bio</label>
<textarea id="bio" name="bio" rows="4"></textarea>
```

Unlike `<input>` , a `textarea` has content and a closing tag. The initial text goes between the opening and closing tags rather than in a `value` attribute. The `rows` attribute is a starting visible height; the user can usually drag the corner to make it bigger.

`<select>` is a dropdown of choices:

html

```
<label for="country">Country</label>
<select id="country" name="country">
  <option value="">Choose a country</option>
  <option value="us">United States</option>
  <option value="ca">Canada</option>
  <option value="uk">United Kingdom</option>
</select>
```

Each `<option>` has its own `value` attribute (what the server receives) and visible text (what the user sees). The first option above has an empty value and acts as a "no selection yet" placeholder. Add `multiple` on the `<select>` to allow more than one selection at once.

`<button>` is the element that submits the form:

html

```
<button type="submit">Sign up</button>
```

The `type` on a button matters more than it looks. Inside a `<form>`, `type="submit"` is the default and submits the form when clicked. `type="button"` is a plain button that does nothing on its own and is meant for JavaScript to wire up. `type="reset"` clears the form but is rarely useful in practice.

If you leave `type` off a `<button>` inside a `<form>`, the browser treats it as a submit button. That is convenient but can cause accidental submissions when you only meant a clickable element. Set `type` explicitly whenever a button is doing something other than submitting.

Browser validation: helpful, but not security

HTML has built-in attributes that ask the browser to check the input before it submits the form:

- `required`: the field must not be empty.
- `minlength` / `maxlength`: bounds on the text length.
- `min` / `max`: bounds on numeric or date values.
- `pattern="[A-Za-z0-9]+"`: a regular expression the value must match.
- `type="email"` / `type="url"`: the browser checks that the value looks like that kind of value.

html

```
<input type="email" name="email" required minlength="3">
```

When the user tries to submit, the browser checks these constraints, shows an error message next to the first bad field, and blocks the submission until everything passes.

That is a UX feature. It is not security. The user can edit the HTML in DevTools, send the request directly from the terminal, or skip the browser entirely. The server must validate every value again, on its own, before doing anything with it. Treat browser validation as a help to honest users, not as a defense against dishonest ones.

 **Always validate on the server** - Anything you check in the browser, you also have to check on the server. The browser runs in the user's

environment, not yours. A determined user can send any data they want, including data that bypasses every HTML constraint on the page.

File uploads

To let users upload files, use `type="file"` . To make the upload actually work, the form needs a special encoding:

html

```
<form action="/upload" method="post" enctype="multipart/form-data">
  <label for="avatar">Profile picture</label>
  <input type="file" id="avatar" name="avatar">
  <button type="submit">Upload</button>
</form>
```

`enctype="multipart/form-data"` tells the browser to send the file as a structured upload rather than as URL-encoded text. Without it, the file's contents are not sent properly. Usually only the file name is sent, which is not what you want.

You can let users pick more than one file with `multiple` , and steer the file picker with `accept` :

html

```
<input type="file" name="photos" multiple accept="image/*">
```

The `accept` attribute is a UX hint: it sets the default filter in the file picker. It does not stop the user from sending other file types. As with browser validation, the server has to check the file type, size, and contents on its own.

What the server actually receives

The clearest way to understand a form is to look at the request the browser sends.

A search form with `method="get"` and two inputs:

html

```
<form action="/search" method="get">
  <input type="text" name="q">
  <input type="number" name="page" value="1">
  <button type="submit">Search</button>
</form>
```

produces a request like:

text

```
GET /search?q=html&page=2 HTTP/1.1
Host: example.com
```

The same form switched to `method="post"` :

text

POST /search HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded

q=html&page=2

The same shape, name and value pairs, moves from the URL to the body. That is the only real difference between the two methods at the HTML level.

If you do not see a `name` attribute on a form control, that control will not appear in either place. Empty input boxes still get submitted (with an empty value) as long as they have a `name` . A label without a `for` , or an input without a `name` , will both submit "successfully" but leave gaps you may not notice until the server complains.

When forms come from app data

Most forms in real apps are wired up to a backend that produces the page and processes the submission. A few habits pay off:

- **Escape user-submitted values when you echo them back.** If you show a comment, name, or search term back on the page, your templating engine should HTML-escape it. Otherwise a value like `<script>alert(1)</script>` becomes a real script tag in someone else's browser. This is the most common cause of cross-site scripting.
- **Use a real form library or framework** for anything beyond a trivial form. Validation rules, error display, repopulating fields after errors, and CSRF tokens are easy to get wrong by hand.
- **Send the right status codes.** A successful `POST` usually returns a redirect to the new page (`303 See Other` or `302 Found`), not a fresh HTML response. This pattern, called Post/Redirect/Get, keeps the back button and the reload button from re-submitting the form.

Try it

Build a small form and watch what the browser actually sends:

html

<!DOCTYPE html>
<html lang="en">
 <head>
 <meta charset="UTF-8">

```
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Form practice</title>
</head>
<body>
  <h1>Sign up</h1>

  <form action="/signup" method="post">
    <p>
      <label for="email">Email</label>
      <input id="email" name="email" type="email" required>
    </p>
    <p>
      <label for="password">Password</label>
      <input id="password" name="password" type="password" minlength="8" required>
    </p>
    <p>
      <label>
        <input type="checkbox" name="newsletter" value="yes">
        Send me product updates
      </label>
    </p>
    <p>
      <button type="submit">Create account</button>
    </p>
  </form>
</body>
</html>
```

Open the page, then open DevTools and switch to the **Network** tab. Tick **Preserve log** so the request stays visible after the page navigates. Fill in the form and submit.

A few things to look at on the resulting request:

- The **method** is **POST** and the path is the form's **action** . Open the request and look at its **Payload** or **Request** tab.
- The payload shows the name and value pairs: **email=...&password=...** . The checkbox only appears if you ticked it. That is the **name** and **value** attributes in action.
- Change **method** to **get** and submit again. The same data now appears in the URL instead of the body.
- Remove the **name** from one of the inputs and submit. That field disappears from the payload entirely, even though the user typed into it.

You will get a network error on submit because **/signup** does not exist on your local file. That is expected. The point is to inspect the request the browser builds. If you want to see a real server's response too, change **action** to **https://httpbin.org/post** . That URL echoes the request back, which is useful when poking at forms.

< 0 / 7 >

✓Knew

0 Items

★Learnt

0 Items

▶Skipped

0 Items

↺ResetProgress

Test Your Knowledge

What does a form do when submitted?

[Click to Reveal the Answer](#)

- ✔ Already Know that
- ✦ Didn't Know that
- ▶ Skip Question



LESSON COMPLETE

Nicely done.

Up next: Pricing Comparison Table

Next Lesson

Containers and semantic layout

Once a page has more than a heading and a few paragraphs, you start needing ways to group elements together. Group a logo and a navigation bar so they form a header. Group an article and a sidebar so they sit next to each other. Group three inputs so you can style them as one card. Almost every layout decision on a real page comes down to deciding which elements belong together.

HTML gives you two kinds of containers for this. **Generic containers** group elements but say nothing about their meaning. **Semantic containers** group elements and also describe what they are: a header, a navigation block, the main content of the page.

You should always reach for the semantic container first, and fall back to a generic one only when nothing more specific fits.

Block versus inline elements

Before containers make sense, it helps to know how HTML elements lay themselves out by default.

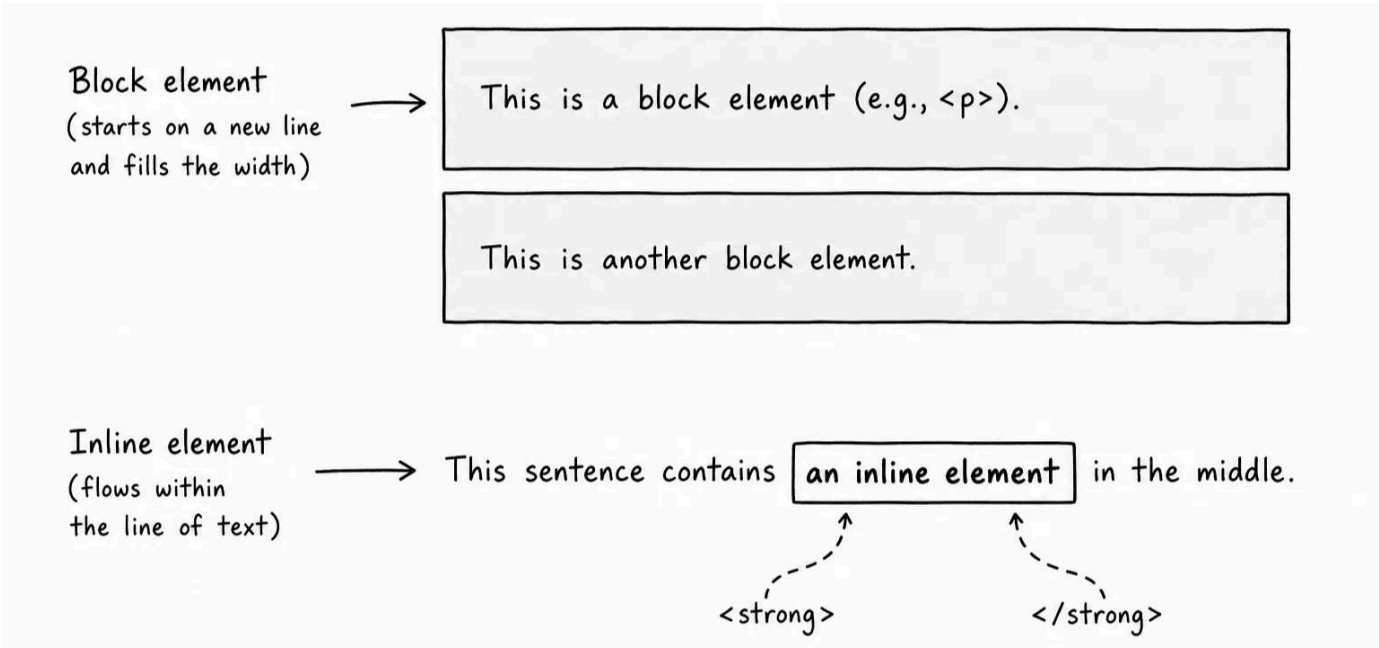
Most elements fall into two layout categories:

- **Block elements** start on a new line and stretch to fill the available width. Headings, paragraphs, lists, and most container elements behave this way.
- **Inline elements** sit inside a line of text without breaking it. `<a>` , `<em>` , `<strong>` , `<code>` , and `<img>` are inline by default.

html

```
<p>This paragraph is a block element.</p>
<p>This sentence contains <strong>an inline element</strong> in the middle.</p>
```

CSS can change any element's display mode, but the default is part of why HTML "just works" before any styling. Block elements form rows. Inline elements flow with the surrounding text.



Generic containers: div and span

When you just need to group elements with no implied meaning, HTML has two generic boxes.

<div> is a generic block container. It is usually used for larger chunks of content.

html

```
<div>
  <p>This paragraph and the next one are inside the same group.</p>
  <p>Anything CSS does to the div applies to both.</p>
</div>
```

**** is a generic inline container. It groups a small piece of text inside a line.

html

```
<p>Hover over <span class="hint">this word</span> to see a tooltip.</p>
```

Neither element means anything on its own. They are there to give you a hook for CSS classes, JavaScript event handlers, or layout adjustments. The browser, the screen reader, and the search engine all treat them as "an element you wanted to group", no more and no less.

Generic containers are useful, but they are also the lazy default. The next sections are about why a more specific container is almost always a better choice.

When generic containers add up: div soup

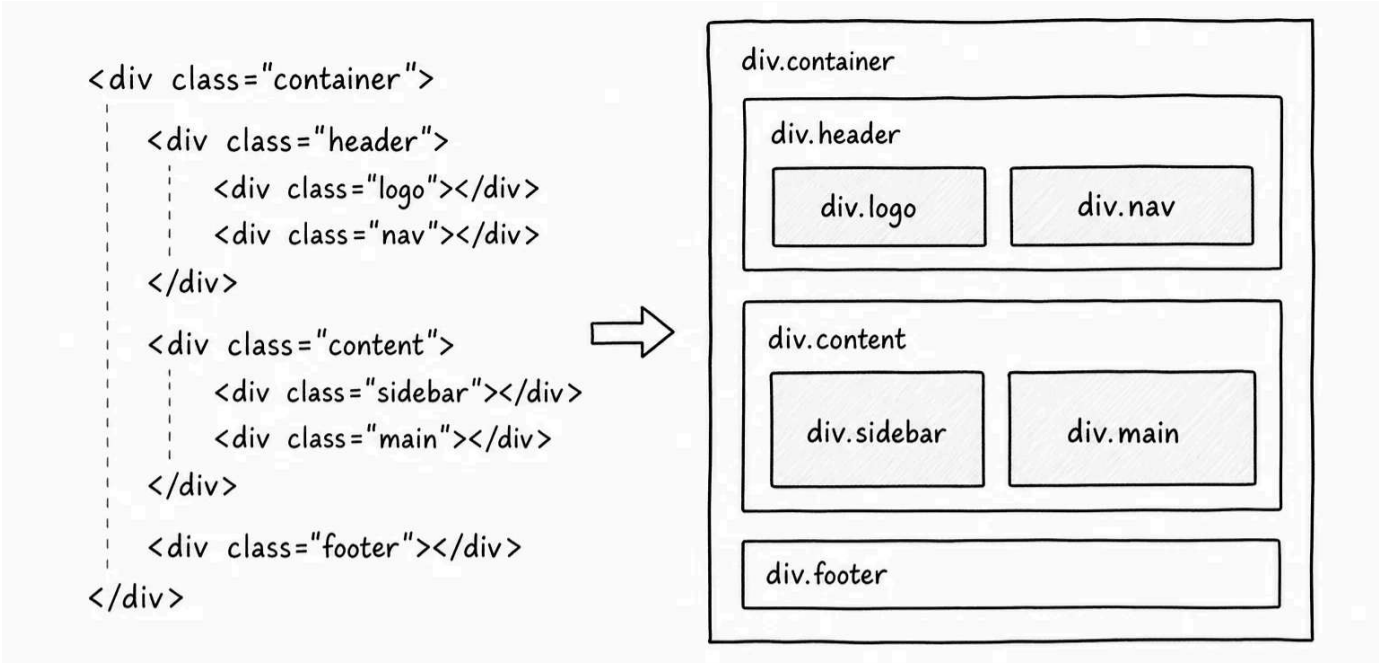
A page written entirely with **<div>** and **** is sometimes called "div soup". Every region is a div. Every navigation menu is a div. Every header, every sidebar, every footer is a div.

html


```
<div class="page-header">
  <div class="logo">...</div>
  <div class="navigation">...</div>
</div>
<div class="page-content">
  <div class="article">
    <div class="article-title">...</div>
    <div class="article-body">...</div>
  </div>
  <div class="sidebar">...</div>
</div>
<div class="page-footer">...</div>
```

A page like this renders fine, and CSS can make it look great. But the structure has lost something important:

- **Screen readers** see one undifferentiated stream of divs. They cannot offer "skip to main content" or "jump to navigation" because there is no main content or navigation element to jump to.
- **Search engines** have to guess what each region is from the class names, which they may or may not get right.
- **Browser features** like reader mode rely on knowing where the main content of the page is. With div soup, they fall back to fuzzy heuristics.
- **Future developers** have to read the CSS to figure out what each div is for.



The class names tell us, the people who wrote the page, what each block is. The semantic HTML elements tell everyone else.

Semantic layout elements

HTML provides named container elements for the parts of a page that show up almost everywhere. They look like divs to CSS, but they carry meaning.

Element	Use it for
<header>	Introductory content at the top of a page or section: logo, page title, intro t
<nav>	A major navigation block: primary nav, table of contents, breadcrumbs

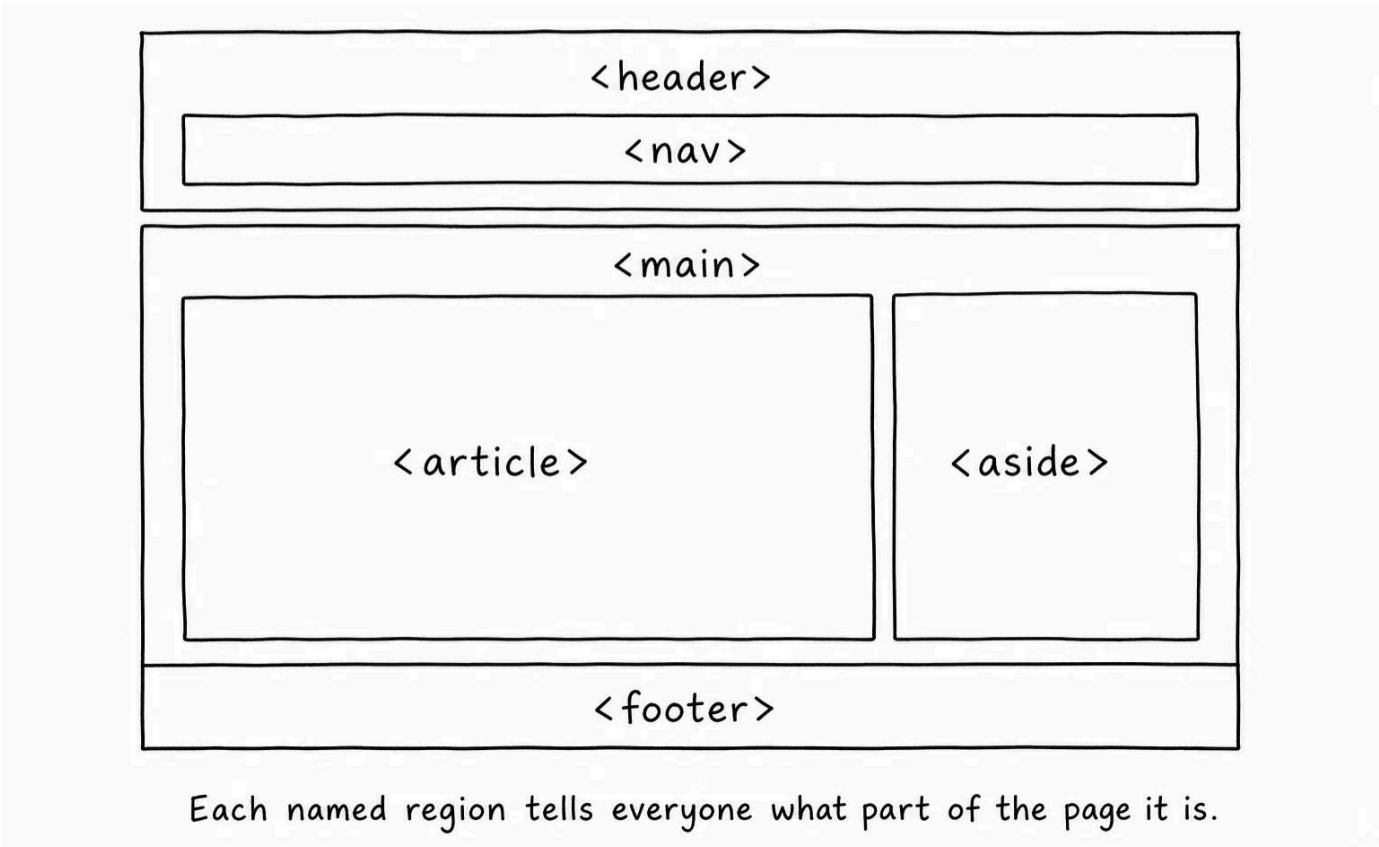
Element	Use it for
<code><main></code>	The single main content of the page; used once per page
<code><section></code>	A thematic group of content, usually with its own heading
<code><article></code>	A standalone, self-contained piece of content: a blog post, a comment, a news item
<code><aside></code>	Content related to the surroundings but not part of the main flow: sidebars, pull quotes, captions
<code><footer></code>	Closing content for a page or section: copyright, contact, related links

Used together, the same div-soup page above becomes:

html

```
<header>
  
  <nav>
    <a href="/">Home</a>
    <a href="/pricing">Pricing</a>
    <a href="/blog">Blog</a>
  </nav>
</header>
<main>
  <article>
    <h1>The article title</h1>
    <p>The article body...</p>
  </article>
  <aside>
    <h2>Related posts</h2>
    ...
  </aside>
</main>
<footer>
  <p>© 2026 Acme. All rights reserved.</p>
</footer>
```

The layout can still look exactly the same, but every container now tells the browser, the screen reader, and the next developer what its job is. Semantic elements do not replace CSS. CSS still does the visual work. The win is structural.



Picking the right semantic element

A few patterns help when deciding which element to use.

`<main>` appears at most once per page. It holds the part of the page that is unique to that route: not the header, not the footer, not the sidebar. Browsers, screen readers, and reader-mode features all use `<main>` as the anchor for "this is the content the user actually came for".

`<article>` and `<section>` look similar but mean different things. An `<article>` is something that would still make sense on its own if you lifted it out and put it somewhere else: a blog post, a news story, a comment, a product card. A `<section>` is a thematic group of content with its own heading inside a larger document. If you cannot give it a clear heading, it is probably a `<div>`, not a `<section>`.

`<nav>` is for major navigation blocks. A short list of links at the bottom of a paragraph does not need to be wrapped in `<nav>`. Use it for navigation that helps users move around the site as a whole. Too many `<nav>` elements weaken the signal that the important ones carry.

`<aside>` is for content related to the surrounding content but not part of its main flow: callouts in the margin, lists of related posts, pull quotes. It is not just "anything on the side of the page". It is content the page could survive without.

When no semantic element fits, fall back to `<div>` or `<span>`. There is nothing wrong with using them; the goal is to use them only when nothing more specific applies.

💡 **Stay specific** - If you find yourself reaching for `<div>`, pause for a second. Is this a header, a navigation block, a section, an article, an

aside, or a footer? If yes, use the right element. If not, `<div>` is fine.

Why semantic HTML matters

Semantic markup is one of the cheapest improvements you can make to a page. The same code that renders identically in the browser also brings:

- **Accessibility.** Screen readers expose semantic regions as "landmarks" that users can jump between with a single keystroke. "Skip to main content" is a real shortcut in many assistive tools, and `<main>` is what makes it work.
- **SEO and crawlers.** Search engines use semantic structure as a hint for what the page is about. Tools that summarize or excerpt pages do the same.
- **Tooling.** Reader modes, browser extensions, screen capture tools, and translation tools all behave more reliably on semantic markup.
- **Maintenance.** A future developer, often you six months from now, can read the page's HTML and immediately see what each region is for, without digging through the CSS.

None of these wins require any new tags you have not already met. They come from

using `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>`, and `<footer>` instead of a generic `<div>`.

 **Use native HTML before ARIA** - ARIA attributes can add accessibility information when native HTML cannot express something clearly. They are not a replacement for good markup. Prefer real elements like `<button>`, `<nav>`, `<main>`, `<label>`, and `<table>` before adding `role` or `aria-*` attributes. Used incorrectly, ARIA can make a page harder to use.



AI Tutor

What are ARIA attributes? >

When layout comes from app data

Most real pages are not hand-written one file at a time. They are built from templates and components. A few habits keep the semantic structure intact:

- **Pick semantic containers at the component level.** A `Header` component should render a `<header>`, not a `<div className="header">`. The naming and the markup should agree.

- **Keep one main element per page.** Even if your layout has several panes, only one of them should be the page's `<main>` content. Components that just wrap their children should usually use a generic element, not `<main>`.
- **Keep heading levels in sync with the structure.** A `<section>` without a heading is usually a sign you need a `<div>` instead. If the template puts a `<section>` on the page, it should also put a real `<h2>` or `<h3>` inside it.

Try it

Save this as `layout.html` and open it in your browser. It is intentionally written with only `<div>` and `<span>`:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Layout practice</title>
  </head>
  <body>
    <div>
      <div>My blog</div>
      <div>
        <a href="/">Home</a>
        <a href="/about">About</a>
      </div>
    </div>
    <div>
      <div>
        <div>An interesting post</div>
        <div>A paragraph of post content.</div>
      </div>
      <div>
        <div>Related</div>
        <div>Links to other posts.</div>
      </div>
    </div>
    <div>© 2026 My blog</div>
  </body>
</html>
```

Now refactor it using the semantic elements from this lesson. Replace the outer top group with `<header>` and put the links inside a `<nav>`. Wrap the body in `<main>`, the post in `<article>`, and the related links in `<aside>`. Wrap the bottom line in `<footer>`. Promote the post title to an `<h1>` and the related-links heading to an `<h2>`.

Open DevTools and look at the **Elements** tab. Click on each region in turn. The browser highlights it on the page and shows its tag name. With the original file, every region was a `div`. With the refactored version, each region now carries its real role.

Knew

0 Items

Learnt

0 Items

Skipped

0 Items

ResetProgress

Test Your Knowledge

What is the difference between block and inline elements?

[Click to Reveal the Answer](#)

Already Know that

Didn't Know that

Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Pricing Comparison Table

Next Lesson

https://roadmap.sh/packs/html/containers-and-semantic-layout

8/8